

# Learning to crawl deep web <sup>☆</sup>



Qinghua Zheng, Zhaohui Wu <sup>\*</sup>, Xiaocheng Cheng, Lu Jiang, Jun Liu

MOE KLINNS Lab and SKLMS Lab, Xi'an Jiaotong University, No. 28, Xianning West Road, Xi'an 710049, China

## ARTICLE INFO

### Article history:

Received 23 June 2011

Received in revised form

11 November 2012

Accepted 7 February 2013

Available online 19 February 2013

### Keywords:

Hidden web

Deep web crawling

Reinforcement learning

## ABSTRACT

Deep web or hidden web refers to the hidden part of the Web (usually residing in structured databases) that remains unavailable for standard Web crawlers. Obtaining content of the deep web is challenging and has been acknowledged as a significant gap in the coverage of search engines. The paper proposes a novel deep web crawling framework based on reinforcement learning, in which the crawler is regarded as an agent and deep web database as the environment. The agent perceives its current state and selects an action (query) to submit to the environment (the deep web database) according to Q-value. While the existing methods rely on an assumption that all deep web databases possess full-text search interfaces and solely utilize the statistics (TF or DF) of acquired data records to generate the next query, the reinforcement learning framework not only enables crawlers to learn a promising crawling strategy from its own experience, but also allows for utilizing diverse features of query keywords. Experimental results show that the method outperforms the state of art methods in terms of crawling capability and relaxes the assumption of full-text search implied by existing methods.

© 2013 Elsevier Ltd. All rights reserved.

## 1. Introduction

The deep web (hidden web) refers to the portion of the World Wide Web that is not part of the surface web, which is directly indexed by search engines. Various studies show that the deep web is particularly valuable. Not only its estimated size is hundreds of times larger than that of the surface web, but also it provides users with high quality information [1–3]. However, to obtain data from the deep web is challenging and has been acknowledged as a significant gap in the coverage of search engines [4]. There are typically two approaches to access the deep web content: (1) virtual-integration approach or “database-centered, discovered-and-forward” access model, and (2) surfacing approach or “crawl-and-index” technique [2,5]. The first approach, also known as federated search [6],

employs the data integration paradigm. In this approach, the user queries are redirected to the relevant source, which is selected by the deep web database selection techniques [7], through a web form based on a mediated schema. Classic works in this direction include WISE-Integrator [8], Meta-Querier [9], and UDI system [10]. Surfacing is another common solution to search the deep web,<sup>1</sup> in which the crawler pre-computes the submissions for deep web forms and exhaustively indexes the response results off-line as other static HTML pages. While the integration approach seems to be a promising way of constructing vertical search engines in specific domains, the surfacing approach is better at large scale and domain independent web search. The surfacing approach leverages the existing search engine infrastructure hence adopted by most of crawlers, such as HiWE (hidden web exposor) [11], hidden web crawler [12] and Google's deep web crawler [4].

One critical challenge in the surfacing approach is how a crawler can automatically generate promising queries

<sup>☆</sup> The premature work of this paper has appeared in Refs. [38,39].

<sup>\*</sup> Corresponding author. Tel.: +86 29 82665262 804.

E-mail addresses: qzhzheng@mail.xjtu.edu.cn (Q. Zheng), laowuz@gmail.com (Z. Wu), cxcaixinster@gmail.com (X. Cheng), roadjiang@yahoo.com (L. Jiang), liukeen@mail.xjtu.edu.cn (J. Liu).

<sup>1</sup> We may use crawl and surface interchangeably in the rest of the paper.

so that it can carry out efficient surfacing. The challenge has been studied by several researches such as Refs. [4,12–16]. In these works, candidate query keywords are generated from the obtained records. Then their harvest rates, i.e., the promise to obtain new records, are calculated according to their local statistics, such as DF (document frequency) and TF (term frequency). The one with the maximum expected harvest rate will be selected for the next query. The basic idea is similar while the difference is that they choose different strategies to derive the estimated harvest rate for each query candidate.

However, to the best of our knowledge, the existing methods suffer from the following deficiencies.

First, the existing methods relied on a critical assumption that full-text search are provided by deep web databases, where all of the words in every document are indexed. They probably work well on deep web databases providing unstructured content with full-text search interface. However, this assumption does not hold in (1) sites with partial full-text index where template words or stop words specified by those sites are excluded and (2) structured databases with only some important fields indexed. Thus, the existing full-text-database estimation techniques can hardly be applied to these databases. For example, the Zipf Law [12,18] assumes there is a connection between the DF rank of a keyword and quantity of response records of that keyword. However, in those deep web sites, a keyword of high DF rank may not even be indexed. Furthermore, treating all deep web pages as pure text documents ignores the semantics in the underlying structured content, i.e., data records with typed fields [19]. For a deep web site providing keyword search box with data type restrictions such as “author”, “city”, or “date”, the semantics of the surfaced data might be helpful. For example, it is more reasonable to submit the keyword “New York City” to the text box “city” rather than to “author”. To overcome this problem, we need to delve deeper in the measurement of promising keywords for a given deep web site. Intuitively, the “semantics” encoded by html tags and the “linguistics” encoded by POS tags could be useful. However, there are no simple rules to distinguish promising index words from non-reward stop words. For example, ‘a’, ‘from’, ‘its’, and ‘any’ can retrieve a large number of records from Citeseer while ‘an’, ‘the’, ‘on’ and ‘this’ match no results. Other sites like Yahoo movie, Baidu Baike, etc., might have their special stop word lists. Thus, how to leverage more valuable information to identify better indexed keywords becomes our first motivation.

Second, the existing methods solely utilize the statistics of acquired data records while ignoring the potential experiences that can be learnt from previous queries. When a crawler issues an unpromising keyword which brings few or even no response records, the statistic of the local data will remain the same. Therefore, it is likely that the crawler will make similar mistakes in its future decisions. For example, suppose the top 3 candidate query keywords are  $\{k_1, k_2, k_3\}$  at the current step based on some statistic like DF and all the three can actually get no response records. After executions of queries using keywords  $k_1$  and  $k_2$ , the DF ranks of all candidate query

keywords do not change, so the next selected query keyword will still be  $k_3$  in spite of the previous lessons of failure. Our intuition is that both the failing and successful experience should benefit the future selection. Suppose after five actions {resources:2, this:899, a:923, four:0, to:960} (number is the reward, or number of new records a query can retrieve) the candidate keywords ranked by DF is {copyright, movie, five, that, in, my}, the DF-based query selector will simply choose “copyright” as the next. However, the “experience” might imply words like “this”, “a” and “to” tend to be much more rewarding than “resource” and “four”. If we take the “experience” into account, “that”, “in” and “my” could be better keywords than “copyright” and “movie” in spite of lower DF rank. Hence, a new challenge is how to adaptively learn the experience from the issued queries to benefit query selection.

Third, query selection decision is made solely based on the immediate reward using statistical information available in local data, so the future reward of each query is ignored, also known as the “near-sighted estimation problem” [17]. This inherent deficiency makes the existing methods fail to look ahead to future steps to make a long-term decision. Suppose we have candidate keywords with their estimated rewards (which are very close to the ground truth) {copyright:500, movie:100}, we choose “copyright” because it is expected to gain more reward in the next query. However, if we look ahead to a further step things become different. Suppose after execution of “copyright” the new candidate set becomes {five:150, movie:100} while choosing “movie” instead of “copyright” makes the new candidate set turn to {that:1000, copyright:500}, then “movie” will bring  $500 + 1000 = 1500$  reward in the next two steps while “copyright” can bring only 650. In this sense, we might consider “movie” as a much more promising keyword than “copyright”. This toy example shows the effect of long-term interest. However, a key issue here is how to accurately and efficiently estimate the reward of the future query after the immediate next.

To address the first and second problems, a crawler need be able to adaptively learn some hidden patterns that distinguish rewarding keywords from non-rewarding ones from the past experience by leveraging multiple valuable features. For the third problem, we need a metric that builds upon future reward. To this end, we present a new framework based on the reinforcement learning [20] for deep web crawling. In this framework, a crawler is regarded as an agent and the deep web database as the environment. The agent perceives its current state and selects an action (query) to submit to the environment according to estimated future reward. The environment responds by giving the agent some rewards (new records) and changing it into the next state. Each action is encoded as a tuple using its linguistic, statistic and HTML features. The rewards of the unexecuted actions are evaluated by their executed neighbors using kNN. Because of the learning policy, a crawler can hopefully give higher priority to rewarding keywords and keep away from unpromising queries, as long as similar queries have been issued. The experimental results on several real world deep web sites show that the reinforcement learning

crawling method relaxes the assumption of full-text search and outperforms existing methods. To sum up, the main contributions of our work are

- We introduce a formal framework for the deep web surfacing problem. To the best of our knowledge, ours is the first work that introduces machine learning approaches to the deep web surfacing problem.
- We formalize the problem in the framework and propose an efficient and applicable surfacing algorithm working well on various deep web sites. The semantics of query form and the structured deep web content can be called into full play under this framework.
- We develop a Q-value approximation algorithm that allows a crawler to select a query by learning from the experience of executed queries. We classify the state-of-art deep web crawling methods into three categories of baselines and demonstrate how our RL method outperforms them.

The rest of this paper is organized as follows: [Section 2](#) gives a brief introduction of related work. [Section 3](#) presents the formal reinforcement learning framework. [Section 4](#) proposes the crawling algorithm and discusses the key issues in it. The experimental results are described in [Section 5](#). The final section draws conclusions and proposes future work.

## 2. Related work

There is a rich literature in the area related to deep web crawling. Bergman [1] and He et al. [2] gave a survey of accessing deep web content. Madhavan et al. [5] gave a more technical perspective on harnessing deep web mainly from two aspects – surfacing and integration. The representative deep web integration works include in Refs. [3,8–10]. In this paper, we focus on the deep web surfacing problem, whose underlying methods can be summarized into two categories: prior knowledge guidance and iterative exploration.

The prior knowledge guidance approach constructs the knowledge base of the target deep web database first, and then generates queries under the guidance of prior knowledge base. For example, Raghavan and Garcia-Molina [21] proposed a deep web crawling method based on Label Value Set (LVS) table, which is used to pass values to query forms as prior knowledge. Alvarez et al. [22] brought forward a method based on domain definitions, which improved the accuracy of filling out deep web forms to some extent. These methods can yield good performance on those deep web forms providing sufficient knowledge, for example the forms containing select-menus like zip codes, city names, dates and prices. It is reported that 6.7% of forms written in English in the United States contain these inputs [5]. However, they are much fewer compared to the text boxes only accepting keywords, which are considered as the main interfaces to surface the deep web content. Wang et al. [15] solved the deep web crawling problem using set-covering sampling based method that only focused on the textual databases

through keywords-based query interfaces. First it downloads a relative small set of sample data records from the target deep web site. Then it constructs a set of queries that can cover most of the sample records. Their empirical study shows that using a sample set of around 2000 documents, they can obtain very few queries (20 in their experiments) covering most of the total data source (more than 90%). However, since their experiments were all conducted on four local corpora by building a full text search engine using Lucene [32], their conclusions may not be applicable to the real world deep web sites, especially to those without full text indexes. If 10 words can cover 90% records of a full-text-indexed movie site, it is less possible that 10 words can get 90% coverage if only titles of movie are indexed. Besides, even if we can build a small number of queries that have a very good coverage of the deep web sites, they may not truly crawl all the covered data since the sites usually limit the number of the returned results, i.e., only a portion of records can be harvested.

The iterative exploration approach does not construct prior knowledge or query sets beforehand. It studies how to interactively submit queries to the deep web form containing text boxes in order to maximize the coverage, based on heuristics and greedy methods. Barbosa and Freire [13] first introduced this idea, and presented a query selection method that generates the next query using the most frequent keywords in the acquired records. However, queries with the most frequent keywords do not ensure that more new records are returned from the deep web database. Ntoulas et al. [12] proposed a greedy query selection method based on the expected harvest rate, which estimates how many new documents a keyword can harvest based on Zipf estimator and the DF of that keyword in the acquired records set. The keyword with the maximum expected harvest rate will be selected for the next query. Madhavan et al. [4] improved the keyword selection algorithm by ranking keywords by their TF-IDF scores. Jiang et al. [16] presented a supervised learning method to evaluate a keyword by its HTML features besides TF and DF. Liu et al. [14] explored the problem on an entire query form by introducing a novel concept MEP (Minimum Executable Pattern). In this method, a MEP set is built and then promising keywords are selected by the joint harvest rate of a keyword and its pattern. By selecting among multiple MEPs, the crawler achieves better results. Wu et al. [17] modeled each web database as a distinct attribute-value graph, based on which, the problem was transferred into finding a weighted minimum dominating set in the graph. A greedy link-based query selection method was proposed to approximate the optimal solution. Noted that in Ipeirotis et al.'s [40] work on the cost model for crawl and query based strategy to retrieve text database, the Iterative set expansion is a query based strategy similar to the deep web query selection. But their focus is on different tasks such as information extraction and database content summary.

In summary, the prior knowledge guidance approach can succeed in sites whose knowledge base is easy to construct, like flight or hotel sites, but the domain

dependence harm its applicability to general search engines. It is impractical for deep web crawlers to gain and maintain the prior knowledge automatically from a large number of deep web sites with complex and dynamic query interfaces. For the web applications such as integration systems or search engines, discovering a target deep web site and then iteratively crawling its contents, although sometimes with higher cost, is a practically better solution.

### 3. Framework

This section proposes a formal framework for the deep web crawling based on the reinforcement learning (RL) and formalizes the crawling problem under the framework. To be readily comprehensible, we first demonstrate the deep web surfacing process to induce the formal definitions and then give a simple example to illustrate those definitions under the formal modeling of the RL framework.

The typical deep web surfacing is an interactive process where the crawler submits queries to the database to harvest new records as many as possible. A deep web database  $D$  contains a set of data records which can only be retrieved by its query form  $F$  (containing text boxes). Deep web crawling aims to harvest data records in  $D$  by iterative queries through the query form  $F$ . Candidate query keywords are generated from the obtained records and the next query will be selected from the candidate set according to some metric estimating a keyword's promise to get new records. Thus, the relation between a crawler and a deep web database can be well illustrated by Fig. 1, which is a typical diagram of the reinforcement learning process. At any given step, an agent (crawler) perceives its state (the acquired data) and selects an action (query). Q-value in reinforcement learning is hired as the metric that estimates long-term reward. The environment (deep web database) responds by giving the agent some reward (new records) and changing the agent into the successor state.

More formally, we have

**Definition 1.** Suppose  $S$  and  $A$  are two sets of states and actions respectively. A state  $s_t \in S$  represents the acquired portion of the deep web database records at the step  $t$ . An action  $a(k) \in A$  ( $a$  for short) denotes a query to the deep web database with the keyword  $k$ , which causes a transition from state  $s_t$  to a successor state  $s_{t+1}$  with the probability  $p(s_{t+1}|a, s_t)$ .

**Definition 2.** The process of deep web crawling is defined as a discrete decision process  $(S, A, P)$  consisting of a set of state  $S$ , a set of actions  $A$  and transition probabilities distribution  $P$ . A crawling process follows a specific issue policy  $\pi: S \rightarrow A$ , which is a mapping from the set of states to the set of actions.

It is a deterministic process since the next state  $s_{t+1}$  is determined by the submitted action  $a$  under state  $s_t$ . Thus  $P$  is subjected to a uniform distribution. So, a policy  $\pi$  is equal to either a state sequence or an action sequence, i.e.,  $\pi(s_t)_{(t=0 \dots)}$  denotes the crawling process under the policy  $\pi$ .

After execution of an action the agent is responded by receiving a set of data records by the environment. The response can be defined as:

**Definition 3.** Suppose  $D$  is the collection of all data records residing in deep web database. After execution of action  $a$  at state  $s_t$ , the response record set  $R(s_t, a) \subseteq D$  represents the collection of data records responded by the environment. Likewise the portion of the new records in the response record set retrieved by action  $a$  at state  $s_t$  is denoted as  $R_{new}(s_t, a)$  ( $R_{new}(s_t, a) \subseteq R(s_t, a)$ ).

Suppose a crawling process follows an issue policy  $\pi$ , the portion of new records in the response records of action  $a$  at state  $s_t$  can be formulated as

$$R_{new}(s_t, a) = R(s_t, a) \setminus \bigcup_{i=1}^{t-1} R(s_i, \pi(s_i)) \quad (1)$$

It is quite reasonable to assume that  $D$  will keep constant during the crawling process, which is a very short time compared to the lifetime of the deep web database. Hence, the response record set of an action is irrelevant to the state, i.e.,  $\forall i, j, R(s_i, a) = R(s_j, a)$ , which we can rewrite it as  $R(a)$ .

There are two important functions in the process. Transition function  $\delta: S \times A \rightarrow S$  denotes the successor state of the given state and action. Reward function  $r(s_t, a)$  is the reward received at the transition from state  $s_t$  to state  $s_{t+1}$  by executing the action  $a$ , i.e., the portion of new records brought by executing  $a_t$ , computed from the following equation:

$$r(s_t, a) = |R_{new}(s_t, a)| / |D| \quad (2)$$

Though in some cases  $|D|$  is either unknown or cannot be obtained beforehand, the absence of the value does not influence the calculation of the reward as they are relative values to rank actions in the same baseline.

The transition of actions has a cost. In the paper, the cost is measured in terms of the time consumed, i.e.,  $\text{cost}(s_t, a) = t_a + t_r \times |R(s_t, a)| + t_d \times |R_{new}(s_t, a)|$ .  $t_a$  is the cost of issuing an action, which includes the query transmitting time and the query processing time by deep web databases;  $t_r$  is proportional to the average time of handling a response record; and  $t_d$  is proportional to the average time of downloading a new result record.

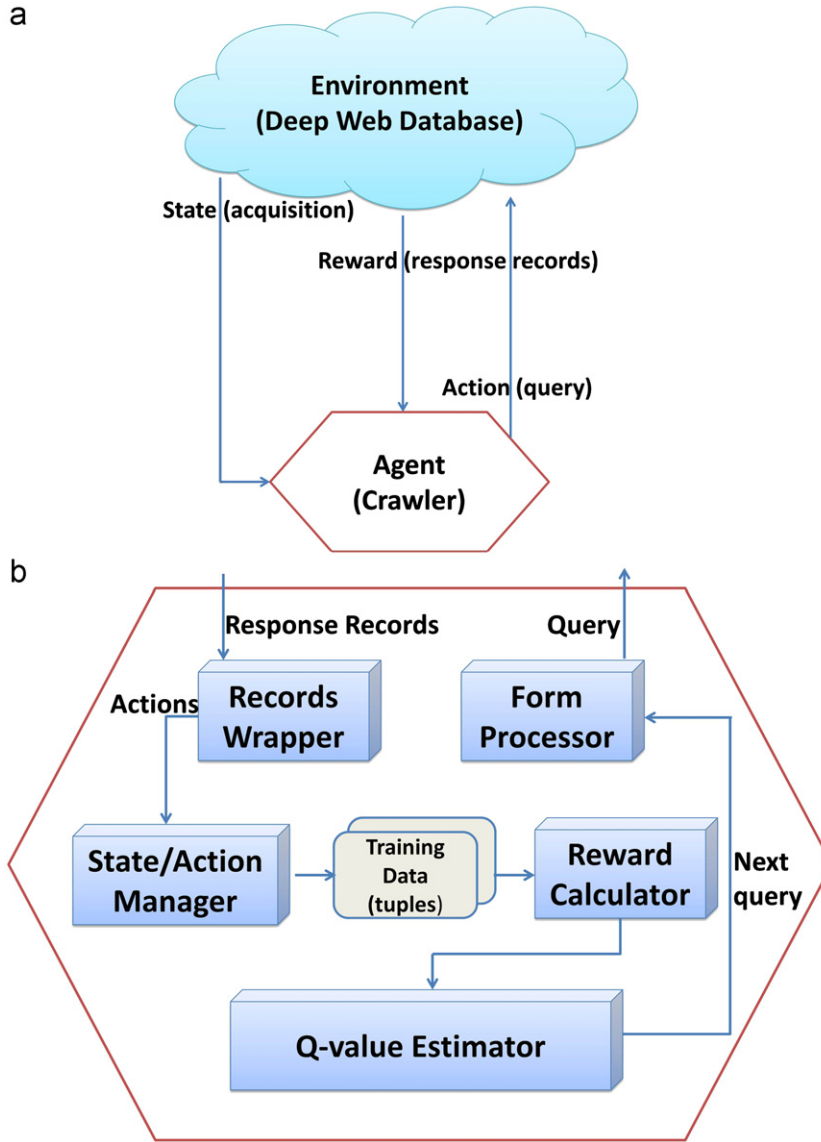
The expectation conditioned on the current state  $s$  and the policy  $\pi$  is called state-value function  $V^\pi(s)$  of states, computed from

$$V^\pi(s_t) = \sum_{i=0}^h \gamma^i r(s_{t+i}, \pi(s_{t+i})) \quad (3)$$

where  $h$  is referred as the step length and  $\gamma$  is the discount factor [23]. Among all policies, there must exist an optimal policy, noted  $\pi^*$  defined as  $V^{\pi^*}(s) \geq V^\pi(s)$  ( $\forall s \in S, \forall \pi$ ). To simplify notations, we write  $V^{\pi^*} = V^*$ .

Based on the presentations above, the formal definition of the deep web crawling problem can be defined as:

**Problem.** Under the constraint  $\sum_{i=0}^{\infty} \text{cost}(s_i, a) \leq \text{cost}_{MAX}$ ,  $\forall s_i \in S$  find such policy  $\pi^* \equiv \arg \max_{\pi} V^{\pi^*}(s_i)$  that maximizes the accumulative reward value. Here  $\text{cost}_{MAX}$  is the maximum cost constraint.



**Fig. 1.** The reinforcement learning framework. (a) Overview of RL framework and (b) elaborate view of Agent in RL framework.

**Example 1.** Suppose  $D$  is  $\{d1, d2, \dots, d10\}$  and  $A$  is  $\{a1, a2, a3, a4\}$ . Suppose the response record set of each action is shown in Table 1,  $R(a1)=\{d1, d2, d3, d7, d9\}$ ,  $R(a2)=\{d2, d3, d4, d6, d10\}$ ,  $R(a3)=\{d1, d2, d5, d8, d9\}$ ,  $R(a4)=\{d1, d2, d3, d6, d7, d10\}$ . The state set  $S=2^D$ . Each possible policy is a permutation of a non-empty subset of  $A$ , making the policy space size reach 64, i.e.,  $P(4,1)+P(4,2)+P(4,3)+P(4,4)$ .

If a crawling process follows a policy  $\pi$  where the action permutation is  $[a1, a2, a3]$ , the state sequence, new records set and reward can be calculated accordingly as shown in the Table 2. Before crawling, the state is empty. After execution of  $a1$ ,  $s_t$  is changed into  $\{d1, d2, d3, d7, d9\}$  and so is the new records set  $R_{new}(s_0, a1)$ . The reward  $r(s_0, a1)$  equals  $|R_{new}(s_0, a1)|/|D|=5/10$ . The next action  $a2$  brings the records set  $\{d2, d3, d4, d6, d10\}$  and causes  $s_t$  become  $\{d1, d2, d3, d4, d6, d7, d9, d10\}$ . The new records

**Table 1**

Queries and their responsive records example.

|      | $d1$ | $d2$ | $d3$ | $d4$ | $d5$ | $d6$ | $d7$ | $d8$ | $d9$ | $d10$ |
|------|------|------|------|------|------|------|------|------|------|-------|
| $a1$ | 1    | 1    | 1    | 0    | 0    | 0    | 1    | 0    | 1    | 0     |
| $a2$ | 0    | 1    | 1    | 1    | 0    | 1    | 0    | 0    | 0    | 1     |
| $a3$ | 1    | 1    | 0    | 0    | 1    | 0    | 0    | 1    | 1    | 0     |
| $a4$ | 1    | 1    | 1    | 0    | 0    | 1    | 1    | 0    | 0    | 1     |

are  $\{d4, d6, d10\}$ . So the reward equals 0.3. The action  $a3$  brings new records  $\{d5, d8\}$  and changes the state into the full records set  $D$ . However, if a different policy is chosen, such as  $[a1, a4, a2, a3]$ , the cost  $(4t_a + 21t_r + 10t_d)$  is larger than that of  $[a1, a2, a3]$   $(3t_a + 15t_r + 10t_d)$ . So  $[a1, a2, a3]$  is a better policy than  $[a1, a4, a2, a3]$ . If the  $cost_{MAX}$  is  $t_a + 6t_r + 10t_d$ , the optimal policy will be  $[a4]$ .



**Table 2**

Crawling process example.

| $t$ | Action | $s_t$                                 | $R_{\text{new}}(s_t, a)$ | $r(s_t, a)$ | $\text{cost}(s_t, a)$ |
|-----|--------|---------------------------------------|--------------------------|-------------|-----------------------|
| 0   | –      | $\phi$                                | –                        | –           | –                     |
| 1   | $a1$   | $\{d1, d2, d3, d7, d9\}$              | $\{d1, d2, d3, d7, d9\}$ | 0.5         | $t_a + 5t_r + 5t_d$   |
| 2   | $a2$   | $\{d1, d2, d3, d4, d6, d7, d9, d10\}$ | $\{d4, d6, d10\}$        | 0.3         | $t_a + 5t_r + 3t_d$   |
| 3   | $a3$   | $D$                                   | $\{d5, d8\}$             | 0.2         | $t_a + 5t_r + 2t_d$   |
| 1   | $a1$   | $\{d1, d2, d3, d7, d9\}$              | $\{d1, d2, d3, d7, d9\}$ | 0.5         | $t_a + 5t_r + 5t_d$   |
| 2   | $a4$   | $\{d1, d2, d3, d6, d7, d9, d10\}$     | $\{d6, d7, d10\}$        | 0.3         | $t_a + 6t_r + 3t_d$   |
| 3   | $a2$   | $\{d1, d2, d3, d4, d6, d7, d9, d10\}$ | $\{d4\}$                 | 0.1         | $t_a + 5t_r + 1t_d$   |
| 4   | $a3$   | $D$                                   | $\{d5\}$                 | 0.1         | $t_a + 5t_r + 1t_d$   |

**Table 3**A toy deep web database  $D$ .

| id  | Title                                                                              | Authors                                                |
|-----|------------------------------------------------------------------------------------|--------------------------------------------------------|
| d1  | The deep web: surfacing hidden value                                               | M.K. Bergman                                           |
| d2  | Annotation of the shallow and the deep web                                         | S. Handschuh and S. Staab                              |
| d3  | Agents and the semantic web                                                        | J. Hendler                                             |
| d4  | Syntactic clustering of the web                                                    | A.Z. Broder, S.C. Glassman, M.S. Manasse, and G. Zweig |
| d5  | A taxonomy of web search                                                           | A. Broder                                              |
| d6  | A comparison of event models for Naive Bayes text classification                   | Andrew McCallum and Kamal Nigam                        |
| d7  | The eyes have it: a task by data type taxonomy for information visualizations      | Ben Shneiderman                                        |
| d8  | An introduction to hidden Markov models                                            | L.R. Rabiner and B.H. Juang                            |
| d9  | A tutorial on hidden Markov models and selected applications in speech recognition | L. Rabiner                                             |
| d10 | The dangers of replication and a solution                                          | J. Gray, P. Helland, P. O’Neil, and D. Shasha          |

In this example where the action set and the database  $D$  are both finite set and have known beforehand, the reward and state-value functions can all be calculated, so the optimal policy can be found out. However, it is impossible to find an optimal policy in the online crawling process. And our goal is to find a best approximation.

#### 4. A motivating example

In this section, we use a more concrete example to introduce the motivation of our solution to the deep web surfacing problem using RL framework. First, we introduce a toy deep web database containing 10 records with two fields, title and authors, as numerated in Table 3. We assume the index of this deep web site is built only on the title field and does not include stop words {the, and, of, for}. We list a sample of words and their responsive records in Table 4 and show the crawling process of DF-based and RL approach in Table 5. A stop word cannot retrieve any records from the database, as shown in the second row of Table 4.

DF-based approach selects the keyword of the highest DF value in local data as the next query. After the initial query “deep”, two new records are harvested. And “the” is selected as the next query because it has the highest DF and TF in the local records. Similarly, “web”, “of”, “and” and “broder” are selected as the following queries. We can see that it does not learn any lesson from the failure of query “the”, but keeps making mistakes on “of” and “and”. Moreover, it cannot avoid keywords like “broder” from the ‘authors’ field.

In the RL approach, each action is encoded as a tuple using its statistic, linguistic, and HTML features. For simplicity, we use a triple in Table 6. Take ‘the: [2, pos7, title]’ as an Example, 2 is the statistical feature DF; pos7

**Table 4**

Selected keywords and their responsive records.

|                     | d1 | d2 | d3 | d4 | d5 | d6 | d7 | d8 | d9 | d10 |
|---------------------|----|----|----|----|----|----|----|----|----|-----|
| {the, and, of, for} | –  | –  | –  | –  | –  | –  | –  | –  | –  | –   |
| Deep                | 1  | 1  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0   |
| Web                 | 1  | 1  | 1  | 1  | 1  | 0  | 0  | 0  | 0  | 0   |
| Hidden              | 1  | 0  | 0  | 0  | 0  | 0  | 0  | 1  | 1  | 0   |
| Value               | 1  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0   |
| A                   | 0  | 0  | 0  | 0  | 1  | 1  | 1  | 0  | 1  | 1   |
| Model               | 0  | 0  | 0  | 0  | 0  | 1  | 0  | 1  | 1  | 0   |
| Agent               | 0  | 0  | 1  | 0  | 0  | 0  | 0  | 0  | 0  | 0   |
| Data                | 0  | 0  | 0  | 0  | 0  | 0  | 1  | 1  | 0  | 0   |
| Broder              | 0  | 0  | 0  | 1  | 1  | 0  | 0  | 0  | 0  | 0   |

indicates the POS feature as listed in the table in the Appendix A; title represents the HTML feature showing the field title of the database. An action in the training set has its reward as its last element. The rewards of the unexecuted actions are evaluated by the executed queries in the training set  $Tr$  using learning algorithms such as kNN. After the query “the”, it learns that keywords with ‘pos7’ will not be rewarding. So words like “of”, “and” and “a” are postponed by words like “hidden” and “model”. Then, it learns a preference to words with ‘title’, thus “broder” from with ‘author’ get little chance than others. By learning from the successful and failure examples like “deep”, “the” and “web”, the crawler makes better choices in the next three query keywords selection. And as more queries are executed, more examples can be used to estimate the reward of new keywords, making the estimator more accurate and more up-to-date. However, more training examples might require more estimating time. We could limit the training set by ruling out the out-of-date actions. The superiority of the long term

**Table 5**  
Crawling process example.

| Method | $t$ | Keyword | $s_t$                            | $R_{\text{new}}(s_t, a)$ | $r(s_t, a)$ | $C$                                |
|--------|-----|---------|----------------------------------|--------------------------|-------------|------------------------------------|
| DF     | 0   | –       | { }                              | –                        | –           | –                                  |
|        | 1   | deep    | {d1,d2}                          | {d1,d2}                  | 0.2         | {the:2,web:2,hidden:1,value:1}     |
|        | 2   | the     | {d1,d2}                          | { }                      | 0           | {web:2,hidden:1,value:1}           |
|        | 3   | web     | {d1,d2,d3,d4,d5}                 | {d3,d4,d5}               | 0.3         | {of:3,and:3,broder:2,hidden:1,a:1} |
|        | 4   | of      | {d1,d2,d3,d4,d5}                 | { }                      | 0           | {and:3,broder:2,hidden:1,a:1}      |
|        | 5   | and     | {d1,d2,d3,d4,d5}                 | { }                      | 0           | {broder:2,hidden:1,a:1}            |
| RL     | 6   | broder  | {d1,d2,d3,d4,d5}                 | { }                      | 0           | {hidden:1,a:1}                     |
|        | 1   | deep    | {d1,d2}                          | {d1,d2}                  | 0.2         | {the, web, hidden, value}          |
|        | 2   | the     | {d1,d2}                          | { }                      | 0           | {web, hidden, value}               |
|        | 3   | web     | {d1,d2,d3,d4,d5}                 | {d3,d4,d5}               | 0.3         | {of, and, broader, hidden, a}      |
|        | 4   | hidden  | {d1,d2,d3,d4,d5,d8,d9}           | {d8,d9}                  | 0.2         | {a, of, and, broder, model}        |
|        | 5   | model   | {d1,d2,d3,d4,d5,d6,d8,d9}        | {d6}                     | 0.1         | {a, of, and, broder, value}        |
|        | 6   | a       | {d1,d2,d3,d4,d5,d6,d7,d8,d9,d10} | {d7,d10}                 | 0.2         | –                                  |

**Table 6**  
Query selection of RL.

| $t$ | Keyword | $Tr$                               | $C$                                                                                                            | Next keyword |
|-----|---------|------------------------------------|----------------------------------------------------------------------------------------------------------------|--------------|
| 0   | –       | { }                                | { }                                                                                                            | –            |
| 1   | deep    | add<br>{deep:[2,pos2,title,0.2]}   | {the:[2,pos7,title],web:[2,pos1,title], hidden:[1,pos2,title],value:[1,pos1,title]}                            | the          |
| 2   | the     | add {the:[2,pos7,title,0]}         | {web: [2,pos1,title],hidden:[1,pos2,title],value:[1,pos1,title]}                                               | Web          |
| 3   | web     | add<br>{web:[2,pos1,title,0.3]}    | {of:[3,pos7,title],and:[3,pos7,title/author],broader:[2,–,author], hidden:[1,pos2,title],value:[1,pos1,title]} | hidden       |
| 4   | hidden  | add<br>{hidden:[1,pos2,title,0.2]} | {and:[5,pos7,title/author],a:[3,pos7,title],of:[3,pos7,title], broader:[2,–,author],model:[2,pos1,title]}      | model        |
| 5   | model   | add<br>{model:[2,pos1,title,0.1]}  | {a:[3,pos7,title], of:[3,pos7,title],and:[5,pos7,title/author], broader:[2,–,author],value:[1,pos1,title]}     | a            |

reward is not shown in this example due to its complexity. More details of the algorithm are presented in the next section.

## 5. Algorithm

This section discusses how to resolve the deep web crawling problem defined in Section 3. There are two crucial factors in solving the problem, i.e., the reward of each action and the Q-value. Section 5.1 introduces the method for the action reward calculation. Section 5.2 presents Q-value approximation and an adaptive algorithm for surfacing deep web. We will continue to use Example 1 to illustrate the definitions and algorithms in this section.

### 5.1. Reward calculation

Before specifying the method for the action reward calculation, it is necessary to have the definition of the document frequency.

**Definition 4.** Suppose on the current state  $s_t$  and the policy  $\pi$ , the document frequency of action  $a(k) \in A$  denoted by  $DF(s_t, a(k))$  ( $DF(s_t, a)$  for short) is the number of documents containing keyword  $k$  in acquired record set  $\cup_{i=1}^{t-1} R(s_i, \pi(s_i))$ .

Note that the document frequency of each action is a known statistic. Since records of  $\cup_{i=1}^{t-1} R(s_i, \pi(s_i))$  having been retrieved at the step  $t$ , the number of documents containing keyword  $k$  can be counted up in the acquired record set. Relying on the Definition 4, the following theorem can be established.

**Theorem 1.** At state  $s_t$ , the reward of each action  $a$  in  $A$  can be calculated from

$$r(s_t, a) = \frac{|R(s_t, a)| - DF(s_t, a)}{|D|}. \quad (4)$$

**Proof.** By incorporating Eq. (1) into Eq. (2), the following equation is born:

$$r(s_t, a) = |R(s_t, a)| - \bigcup_{i=1}^{t-1} R(s_i, \pi(s_i)) / |D|. \quad (5)$$

Eq. (5) can be further rewritten as

$$r(s_t, a) = \frac{|R(s_t, a)| - |R(s_t, a) \cap \cup_{i=1}^{t-1} R(s_i, \pi(s_i))|}{|D|} \quad (6)$$

The intersection in Eq. (6) denotes the collection of documents containing the keyword of action  $a$  in the data set  $\cup_{i=1}^{t-1} R(s_i, \pi(s_i))$ . According to the Definition 4 the value equals to the document frequency of the action, i.e.,

$$|R(s_t, a) \cap \bigcup_{i=1}^{t-1} R(s_i, \pi(s_i))| = DF(s_t, a) \quad (7)$$

Consequently Eq. (4) could be proved by incorporating Eq. (7) into Eq. (6).

Considering the Table 2 of Example 1, under policy  $[a1, a2, a3]$ , we can easily figure out that  $|R(s_1, a2)|=5$ ,  $DF(s_1, a2)=\{|d2, d3|\}=2$ ,  $r(s_1, a2)=(5-2)/10=0.3$ . In the real world deep web crawling process, the absence of  $D$  in Eq. (4) does not affect the final result for the same reason described in Section 3. According to Eq. (4) for an executed action, as response record set  $R(s_t, a)$  is acquired, the reward can be calculated. In contrast, the reward calculation for an unexecuted action directly through Eq. (4) is infeasible. Nevertheless, the response record set of an unexecuted action can be estimated by generalizing from those executed. Before proceeding any further, it is necessary to define the action training and candidate set.

**Definition 5.** Suppose at state  $s_t$ , training set  $Tr$  is a set of executed actions,  $|Tr|=t$ . Similarly, candidate set  $C$  is a set of available action candidates for submission in the current state. Each action in either  $Tr$  or  $C$  is encoded in the same vector space.

Based on Definition 4, for an action  $a_i$  in  $C$ , its reward can be estimated as

$$|\hat{R}(s_t, a_i)| = \sum_{a_j \in Tr} \kappa(a_i, a_j) |R(s_t, a_j)|. \quad (8)$$

where  $\kappa(a_i, a_j)$  is a kernel function used to evaluate the distance between the given two actions, and  $|\hat{R}(s_t, a_i)|$  is the estimated value of  $|R(s_t, a_i)|$ . Since the response record set  $R(s_x, a)$  of an action is irrelevant to the state  $s_x$ , the response record set can be rewritten to the current state  $s_t$ , i.e.,  $R(s_x, a)=R(s_t, a)$ . Accordingly, the intuition behind (8) is to estimate the reward for an action in  $C$  by evaluating those in  $Tr$ . As all response record sets of executed action are at the current state, the size of response record set of an action in  $C$  can be learnt from those sharing the similar features in  $Tr$ . Once the size of response record set of an unexecuted is calculated in (8), the value can then be applied to (4) to calculate its reward.

Now the action rewards for both executed and unexecuted actions can be calculated from (4). In the rest of the subsection, we will discuss how to calculate kernel function  $\kappa(a_i, a_j)$  in (8). Calculating the similarity of actions requires encoding them into a feature space. We incorporate three types of features, linguistic features, statistical features and HTML features to establish the feature space [16].

Linguistic features consist of POS (Part of Speech), length and language of a keyword (action). Length is the number of characters in the keyword. Language represents the language that a keyword falls into. It takes effect in multilingual deep web database.

Statistical features include TF (term frequency), DF (document frequency) and RIDF (residual inverse document frequency) of a keyword in the acquired records. The value of RIDF is computed as

$$RIDF = \log(1 - e^{-TF/|D|}) - \log(DF/|D|) \quad (9)$$

RIDF tends to highlight technical terminology, names, and good keywords and to exhibit nonrandom distributions over documents [24].

The HTML format usually plays an important role in indicating the semantics of the presented data. This brings us to consider the HTML information of keywords. We propose two HTML features tag-attribute and location. Tag-attribute feature encodes HTML tag and attribute information of a keyword, and location represents the depth of the keyword's node in the DOM tree [35] derived from the HTML document. The features may imply the semantic information of a keyword hence is useful in distinguishing unpromising keywords. Take the CiteSeer record page as an example, the words under the key "content" in  $\langle \text{meta name}=\text{"citation\_title"} \text{ content}=\text{"Fast Author Name Disambiguation in CiteSeer"} \rangle$  are probably promising queries while words enclosed in  $\langle \text{div id}=\text{"footer"} \rangle \dots \langle / \text{div} \rangle$  are more likely to be template words which are excluded from the index of the site.

For linguistic and HTML features whose values are discrete, the linear kernel is assigned. Considering that value of statistical feature tends to a Gaussian distribution over documents [24], the Gaussian kernel is adopted to evaluate similarity upon the statistical features, which is formulated as

$$\kappa_s(a_i, a_j) = \exp(-||a_i - a_j||^2 / 2\delta^2). \quad (10)$$

The final kernel function is hybrid of these kernels. Suppose  $\lambda_l$ ,  $\lambda_h$  and  $\lambda_s$  ( $\lambda_l + \lambda_h + \lambda_s = 1$ ) are weights for linguistic, HTML and statistical kernel, respectively, the kernel function to evaluate similarity of two actions is

$$\kappa = \lambda_l \kappa_l + \lambda_h \kappa_h + \lambda_s \kappa_s. \quad (11)$$

In experiments the weight of statistical features usually accounts for a larger part. The details for the features will be discussed in the feature analysis experiments in Section 6.3.

## 5.2. Q-value approximation and surfacing algorithm

It seems a promising solution to greedily choose an action with maximum reward at each step for deep web crawling problem. However, it cannot guarantee a global optimal policy since a secondary action at current state may bring more rewarding actions for future steps than the action with maximum reward. The reinforcement learning theory suggests that it may bring better results while future rewards are considered. We thus present the Q-value estimation method, which takes into account the future reward for action selection.

Once the reward of each action is obtained, given the problem definition in Section 3, the agent can find an optimal policy  $V^*$  if the  $V^\pi$  of each state can be calculated. The calculation of  $V^\pi$  could be well solved when the agent uses the Q-function [25,26]

$$V^\pi(s_t) = Q(s_t, \pi(s_t)) = r(s_t, \pi(s_t)) + \gamma \max[V^\pi(\delta(s_t, \pi(s_t)))]. \quad (12)$$

Here Q-function  $Q(s, a)$  represents the reward received immediately upon executing an action  $a$  from state  $s$ , plus the value discounted by  $\gamma$  thereafter. Using the Eq. (3), the



Q-function can be rewritten as

$$Q(s_t, a) = r(s_t, a) + \max \left[ \sum_{i=1}^h \gamma^i \times r(s_{t+i}, \pi(s_{t+i})) \right]. \quad (13)$$

To simplify the notion, here we let  $\gamma=1$ , i.e., the reward for the future steps are regarded as important as those for the present.  $h$  is a critical parameter denoting the step length looking ahead to the future reward. If  $h=0$ , the future reward is ignored and  $Q$ -value equals to the immediate reward, i.e.,  $Q(s_t, a) = r(s_t, a)$ . When  $h > 1$ ,  $Q$ -value represents the long-term reward. However, as the action reward at state  $s_{t+1}$  is unseen at state  $s_t$ , the  $Q$ -value has to be approximated. To estimate the  $Q$ -value, we make the following assumption: assume at the current state, the action set  $A$  will not enlarge in the next  $h+1$  steps ( $h \ll |A|$ ). When  $h$  is not very large the assumption is reasonable. Under the assumptions, Theorem 2 could be established.

**Theorem 2.** At state  $s_t$  when  $h=1$  the  $Q$ -value of an action  $a_i (a_i, a_j \in C, a_i \neq a_j)$  can be estimated as

$$Q(s_t, a_i) \propto r(s_t, a_i) + \max_j \left[ r(s_t, a_j) + \frac{DF(s_t, a_j)}{|D|} - \frac{|\cup_{i=1}^{t-1} R(s_i, \pi(s_i))| |R(s_t, a_j)|}{|D|^2} \right] \quad (14)$$

**Proof.** To simplify the notion, let  $\cup_{i=1}^{t-1} R(s_i, \pi(s_i)) = R_{t-}$ ,  $R(s_t, a_i) = R_t$ ,  $R(s_{t+1}, a_j) = R_{t+1}$ . First of all, because the action set will not enlarge, the optimal  $Q$ -value can be searched in the action set at the current state. According to (13), when  $h=1$  the  $Q$ -value can be formulated as

$$Q(s_t, a_i) = r(s_t, a_i) + \max_j [r(s_{t+1}, a_j)]. \quad (15)$$

Following the method described in Section 4,  $r(s_t, a_i)$  can be calculated, whereas  $r(s_{t+1}, a_j)$  is unknown at state  $s_t$ . Therefore Eq. (15) is rewritten as

$$Q(s_t, a_i) = \max [|(R_t \cup R_{t+1}) \setminus R_{t-}| / |D|]. \quad (16)$$

Because the response records are independent with each other, the capture-mark-recapture [27] method can be applied to estimate the overlaps records

$$|(R_{t-} \cup R_t) \cap R_{t+1}| / |R_{t+1}| \propto |R_{t-} \cup R_t| / |D|. \quad (17)$$

Further Eq. (17) can be transformed into

$$|(R_{t+1} \cap R_t)| - |R_{t+1} \cap R_{t-}| \propto |R_t \cup R_{t-}| |R_{t+1}| / |D| - |R_{t+1} \cap R_{t-}|. \quad (18)$$

By incorporating Eq. (18) in to Eq. (16) the following equation is born

$$Q(s_t, a_i) = 1 / |D|$$

**Table 7**

Crawling status after the first action a1.

| $t$ | Action | $R(s_t, a)$      | $s_t$            | $r(s_t, a)$ | Tr     | C          |
|-----|--------|------------------|------------------|-------------|--------|------------|
| 0   | –      | $\phi$           | $\phi$           | –           | $\phi$ | $\phi$     |
| 1   | a1     | {d1,d2,d3,d7,d9} | {d1,d2,d3,d7,d9} | 0.5         | {a1}   | {a2,a3,a4} |
| 2   | –      | –                | –                | –           | –      | –          |

$$\times \max [ |R_t \setminus R_{t-}| + |R_{t+1} \setminus R_{t-}| + |R_{t+1} \cap R_{t-}| - |R_t \cup R_{t-}| |R_{t+1}| / |D| ] \quad (19)$$

Note that according to the characteristics of response record set in Definition 3 and Eq. (5)

$$|R(s_{t+1}, a_j) \setminus R_{t-}| = |R(s_t, a_j) \setminus R_{t-}| = r(s_t, a_j) \times |D|. \quad (20)$$

Following (5), (19) and (20) can be reformulated as

$$Q(s_t, a_i) = r(s_t, a_i) + \max_j [ |R_{t+1} \cap R_{t-}| / |D| + r(s_t, a_j) - |R_t \cup R_{t-}| \times |R_{t+1}| / |D|^2 ] \quad (21)$$

Then the Theorem 2 can be derived by incorporating Eq. (7) into Eq. (21).

As all the factors in (14) can be calculated, the  $Q$ -value of each action can be approximated based on the acquired data set. Now, we can approximate  $Q$ -value with a given step length by iteratively applying (14). Note if  $h$  goes too big, the assumption may not hold and the future state may diverge from the experienced state rendering the approximation for future reward imprecise.

We develop an adaptive algorithm for deep web surfacing based on the framework, as shown in Algorithm 1. The algorithm takes the current state and last executed action as input and outputs the next optimal action.

Specifically, the surfacing algorithm first calculates the reward of the last executed action and then updates the action set through Step 2 to Step 7, which causes the agent to transit from its states<sub>t</sub> to the successor state  $s_{t+1}$ . Then the training and candidate set are updated in accordance with the new action set in Step 9. After that the algorithm estimates the reward and  $Q$ -value for each action in candidate set in Step 10 and Step 11. The action that maximizes  $Q$ -value will be returned as the next to be executed action.

#### Algorithm 1. Adaptive RL surfacing algorithm

---

**Input:**  $s_t, a_m$       **Output:**  $\pi(s_{t+1})$

- 1: calculate the reward of action  $a_m$  following Eq. (4);
- 2: for each document  $d_i \in R(s_{t-1}, a_m)$
- 3:    for each keyword  $k$  in  $d_i$  do
- 4:     if action  $a(k) \neq A$  then  $A = A \cup a(k)$ ;
- 5:     else then update TF and DF of action  $a(k)$ ;
- 6:    end for
- 7: end for
- 8: change the current state to  $s_{t+1}$ ;
- 9:  $Tr = Tr \cup \{a_m\}$ ; update candidate set  $C$ ;  $C = C \setminus \{a_m\}$ ;
- 10: for each  $a_i \in C$  update its reward using Eq. (8) and Eq. (4);
- 11: for each  $a_i \in C$  calculate its  $Q$ -value using Eq. (14);
- 12: return  $\arg\max_a [Q(s_t, a)]$ ;

---

**Example 2.** Suppose all the settings are the same as those in Example 1. Let a1 be the initial action. Table 7 shows

the crawling status after execution of  $a_1$ . New documents  $\{d_1, d_2, d_3, d_7, d_9\}$  are harvested. The document  $d_1$  then brings new actions  $a_3$  and  $a_4$ . And  $d_2$  brings  $a_2, a_3$  and  $a_4$  (Table 1), making the action candidate set become  $\{a_2, a_3, a_4\}$ . In Table 8, we then show the reward calculation of each candidate action. Here the true value of  $R(a)$  and  $r(s_1, a)$  are listed, while for the real world deep web crawling they need to be estimated by Eq. (8) and calculated by Eq. (4) respectively. Suppose  $\kappa(a, a_1)$  for each candidate action is assigned in the fifth column of Table 8, then  $|\tilde{R}(a)|$  can be calculated through  $\kappa(a, a_1) * |R(a_1)|/Z$  and  $\tilde{r}(s_1, a) = (|\tilde{R}(a)| - DF(s_1, a))/|D|$ . For example,  $\tilde{r}(s_1, a_2) = (0.7 * |R(a_1)| - DF(s_1, a_2))/|D| = (0.7 * 5 - 2)/10 = 0.15$ .

If we do not consider future reward and choose the action with the maximum estimated reward for the following step, then  $a_2$  will be the next action. After  $a_2$ , the training set will change into  $\{a_1, a_2\}$  and the candidate set will become  $\{a_3, a_4\}$ .

When looking ahead one step for future reward, the  $Q$ -value for the candidate action at  $t=2$  can be estimated using Theorem 2. Here we rewrite it as  $Q(s_t, a_t) \propto r(s_t, a_t) + \max(q(s_t, a_j))$  for short. Table 9 presents the details of the  $Q$ -value calculation. Considering the candidate action  $a_2$ , the next action after  $a_2$  may be  $a_3$  and  $a_4$ . To calculate  $Q(s_1, a_2)$ , we must first calculate  $q(s_1, a_3)$  and  $q(s_1, a_4)$ .  $q(s_1, a_3) = \tilde{r}(s_1, a_3) + DF(s_1, a_3)/|D| - |R(s_1, a_1)| * |R(s_1, a_3)|/|D|^2 = 0.10 + 0.3 - 0.25 = 0.15$ . We can also get  $q(s_1, a_4) = 0.20$ . Thus  $Q(s_1, a_2) = 0.15 + 0.20 = 0.35$ . Similarly, we can calculate the  $Q$ -value for  $a_3$  and  $a_4$ . Finally, the next action will be either  $a_2$  or  $a_4$ .

Based on the example above, we can also conclude that the time complexity of an action selection will grow exponentially as  $h$  increases. The number of times for  $Q$ -value calculation at the current step equals  $\prod_{i=0}^h (|C| - i)$ . When  $h > 0$ , the time complexity of the adaptive RL surfacing algorithm will be much larger than all the existing methods, which only consider immediate reward. For the real world crawling applications, especially the large scale deep web databases, where size of the action candidate set may achieve 1000, considering the future reward will not be efficient.

## 6. Experiments

To demonstrate the effectiveness and efficiency of our proposed approach for deep web crawling, we evaluate our algorithm on 7 real world deep web databases in different scales and domains. We introduce the experimental setup in Section 6.1, present the experimental results on the deep web sites in Section 6.2, and conduct feature analysis in Section 6.3. In Section 6.4, we first

classify the state-of-art deep web crawling methods into three categories of baselines and demonstrate how our reinforcement learning method can degenerate to these baselines. And then we compare the RL method with baseline methods and delve into the crawling logs to analyze the superiority and imperfection of the RL method. The running time analysis and experiments on full text datasets are given in Section 6.5 and Section 6.6 respectively. Finally, the practical suggestions for choosing deep web crawling methods are discussed in Section 6.7.

### 6.1. Experimental setup

To the best of our knowledge, there are no public datasets or even widely acknowledged deep web sites for deep web crawling evaluation. We try to select the well-known real world deep web sites covering different domains, scales, and languages. AbeBooks is a typical e-commerce website. Wikicfp is a well-known “call for paper” site in academia. Paper Open provides archive and search for Open Access papers with multiple languages. Yahoo movie and Google music are the representatives of the movie and music sites. Baidu Baiku is the largest wikipedia-style site in Chinese. CiteSeer is a large scale academic digital library. The detailed information about these databases is listed in Table 10. Here the size of a deep web site indicates the total number of data records residing in it. Notice that the estimated sizes listed in the last column were obtained from displaying statistics in the websites or by calculation beforehand. For example, in AbeBooks each sub-category shows its total number of book items. However, due to the dynamic update of deep web sites, these numbers may not accord with the latest ones.

In the case of AbeBooks, as it is large scale, the agent was restricted to crawl the “historical fictions” category to accelerate the experiment. In Paper Open, it was restricted to “computer science paper”. Queries to the site are applied to the textbox “keywords”. Regarding Wikicfp, Yahoo movie, which are the typical medium deep web databases, we utilize the only generic search

**Table 9**

$Q$ -value caculation for candidate actions ( $h=1$ ).

| Action | $\tilde{r}(s_1, a)$ | Next action | $DF(s_1, a)/ D $ | $q(s_1, a)$ | $Q(s_1, a)$ |
|--------|---------------------|-------------|------------------|-------------|-------------|
| $a_2$  | 0.15                | $a_3$       | 0.3              | 0.15        | 0.35        |
|        |                     | $a_4$       | 0.4              | 0.20        |             |
| $a_3$  | 0.10                | $a_2$       | 0.2              | 0.10        | 0.30        |
|        |                     | $a_4$       | 0.4              | 0.20        |             |
| $a_4$  | 0.10                | $a_2$       | 0.2              | 0.10        | 0.35        |
|        |                     | $a_3$       | 0.3              | 0.25        |             |

**Table 8**

Reward caculation for candidate actions.

| Action | $R(a)$                                | $DF(s_1, a)$ | $r(s_1, a)$ | $\kappa(a, a_1)$ | $ R(a) $ | $\tilde{r}(s_1, a)$ |
|--------|---------------------------------------|--------------|-------------|------------------|----------|---------------------|
| $a_2$  | $\{d_2, d_3, d_4, d_6, d_{10}\}$      | 2            | 0.3         | 0.7              | 3.5      | 0.15                |
| $a_3$  | $\{d_1, d_2, d_5, d_8, d_9\}$         | 3            | 0.2         | 0.8              | 4.0      | 0.10                |
| $a_4$  | $\{d_1, d_2, d_3, d_6, d_7, d_{10}\}$ | 4            | 0.2         | 1.0              | 5.0      | 0.10                |

**Table 10**

Summary of deep web sites used for evaluation.

| DB Name      | URL                        | Domain          | Estimated size |
|--------------|----------------------------|-----------------|----------------|
| AbeBooks     | www.abebooks.com           | Book            | 110,000        |
| Wikicfp      | www.wikicfp.com            | Conference      | 5,200          |
| Paper Open   | www.paperopen.com          | Paper           | 743,000        |
| Yahoo movie  | movies.yahoo.com/mv/search | Movie           | 128,000        |
| Google music | www.google.cn/music/       | Music           | 110,000        |
| Baidu Baike  | Baike.baidu.com            | Wikipedia       | 1,000,000      |
| CiteSeer     | citeseerx.ist.psu.edu      | Digital Library | 1,121,821      |

text boxes as their query interfaces. As for Baidu Baike and Google Music, the sites are multilingual sites consisting of both English and Chinese. On the sites we select the rewarding textbox “Keyword Tag” and “Singer” as their query interfaces. However, restriction on certain search text boxes is not an indispensable condition, but reduces a lot of unnecessary crawling costs and provides accordant criteria for comparison with baseline methods.

A key problem is the setting of action candidate set  $C$ . While training set  $Tr$  store actions that have been issued, candidate set  $C$  maintains actions that are candidate for the next query. Along with the crawling, the training set might grows. It is not necessary to use online learning as the size of  $Tr$  is usually limited within hundreds. In principle, every keyword appearing in every infinite domain element, which constitutes the vocabulary of the database, should be evaluated for the next submitted keywords. Nevertheless, due to the tremendous quantity of keywords, the solution is infeasible in practical. According to Heap’s law [28], the vocabulary size  $M$  (number of unique keywords) is a function of the number of total words  $N$  in the document set, i.e.,  $M = kN^\beta$ , where  $k$  and  $\beta$  are parameters. With English text corpora, typically  $k$  is between 10 and 100, and  $\beta$  is between 0.4 and 0.6 [30]. It was reported that the average web page contained 474 words [31]. So we suppose each deep web record contains 500 words, when  $k=50$  and  $\beta=0.5$  the vocabulary size of a small-scale database  $|D|=10,000$  could grow to  $M=50*(500*|D|)^{0.5}=111,803$ . It is considerably inefficient either to evaluate or to maintain the results in such large vocabulary at each query state. Based on the experimental observations, the candidate set has not to be very large to achieve a satisfactory coverage. Instead of exhaustively evaluating all keywords, we may pick out some “promising” keywords before the evaluation process is actually performed. However, it leaves us two questions: what criteria should be followed to generate a candidate set and how large the candidate set should be to embrace sufficient “promising” keywords?

## 6.2. Effectiveness of RL method

To start our crawling experiments, it is important to first specify what features can be used as criteria to distinguish “promising” keywords. As were shown in the related work, statistical features such as TF and DF are usually employed as indicators of a keyword’s expected reward. However, we use RIDF (Eq. (9)) since it tends to highlight technical terminology, names, and

**Table 11**

Candidate set size settings for RL method.

| DB Name      | $k, \beta, n$ | Estimated size of candidate set | Size of candidate set |
|--------------|---------------|---------------------------------|-----------------------|
| AbeBooks     | 100, 0.4, 300 | 416                             | 500                   |
| Wikicfp      | 100, 0.4, 100 | 368                             | 500                   |
| Paper Open   | 100, 0.4, 500 | 536                             | 1000                  |
| Yahoo movie  | 100, 0.4, 300 | 442                             | 500                   |
| Google music | 100, 0.4, 300 | 416                             | 500                   |
| Baidu Baike  | 100, 0.4, 500 | 603                             | 1000                  |
| CiteSeer     | 100, 0.4, 500 | 632                             | 1000                  |

good keywords and to exhibit nonrandom distributions over documents. The better the criterion is, the smaller the candidate set can be. However, the candidate set cannot be too small to lose some potential promising keywords. Here, we use the average vocabulary size to estimate the candidate size, i.e.,  $size = M/n = kN^\beta/n$ , where  $n$  is the expected number of queries. And the actual size is set to the nearest 500  $m$  ( $m > = 1$ ) bigger than size. Take the AbeBooks as an example,  $N=110000*500$ , if we choose  $k=100$ ,  $\beta=0.4$ , and  $n=300$  (which means that we expect to crawl AbeBooks with 300 queries), the estimated candidate size is 416, so the actual size will be set to 500. Based on this strategy, we present the setting of the candidate set sizes in Table 11.

Our interest is to discover their records as many as possible with affordable cost. To make the results more intelligible, we roughly use *harvest* (Second column in Table 12), i.e., the number of actual retrieved records and *number of queries* (Fourth column in Table 12) to evaluate the crawling effects. Table 12 shows that our method is quite efficient. In the first five cases the agent achieves around 80% coverage by issuing around 500 queries. For the Baidu Baike, its size reaches million and each record of the site is correlated with a particular word entry, so it measures up to a remarkable performance when retrieving 82% coverage by 1959 queries. Moreover, the experimental results indicate that the strategy for setting the candidate set size is applicable.

### 6.3. Features analysis

When encoding an action, three kinds of features are employed, namely the linguistic features, statistical features and HTML features. Our hypothesis is that those features can represent an action's capability of obtaining records, in terms of number of the response data records. In the experiments, feature selection and weight setting may vary from site to site. For example, if the site is in single-language, the language feature will not be hired. The following are the technical details of extracting and setting of these features.

HTML features include tag-attribute and location. The tag-attribute feature encodes the HTML tag and attribute information of a keyword. The location represents the depth of the keyword's node in the DOM tree derived from the HTML document. In experiments, we set a four dimensions vector (*tagName*, *tagClass*, *tagID*, *depth*) to encode the HTML features of a keyword, the first three are numerical and the fourth is integral. For example, in CiteSeer, *tagName* ∈ {"div", "a", "b", "h1", "p", other}, *tagClass* ∈ {"primaryheader", "citation remove", "char\_increased char\_indented char6 padded", "char\_increased char\_indented char\_mediumvalue padded", "para4",

other}, *tagID* ∈ {"conclusion", "main\_content", "introduction"}, and *depth* ∈ [1,11].

Statistical features employed in the experiments include DF rank value and RIDF. The two values are normalized by the following strategies. Suppose the *DFrank* represents the rank of a keyword's DF in the candidate set *C*, the normalization value  $DFrank\_norm = 1 - DFRank/|C|$ . Normalizing RIDF need first calculate the minimum and maximum RIDF value in the candidate set. Then  $RIDF\_norm = (RIDF - \min RIDF) / (\max RIDF - \min RIDF)$ .

Linguistic features include POS, word length and language. POS (Part of Speech) is processed with aid of Stanford Log-linear Part-Of-Speech Tagger [33,34] and encoded as 8 Boolean dimensions. The details are listed in Appendix A. The word length feature is set to 5 bits vector (*v1*, *v2*, *v3*, *v4*, *v5*). Each bit will set to 1 and others -1 if one of the five corresponding criteria (*len*=1, *len*=2, *len*=3, *len*=4, *len* ≥ 5) is met. Language is numerical.

To analyze the importance of the features, we first conduct the RL method using kNN by removing each feature on Yahoo Movie and examine how it shrinks the original RL crawling result curve. The experimental results are shown in Fig. 2(a).

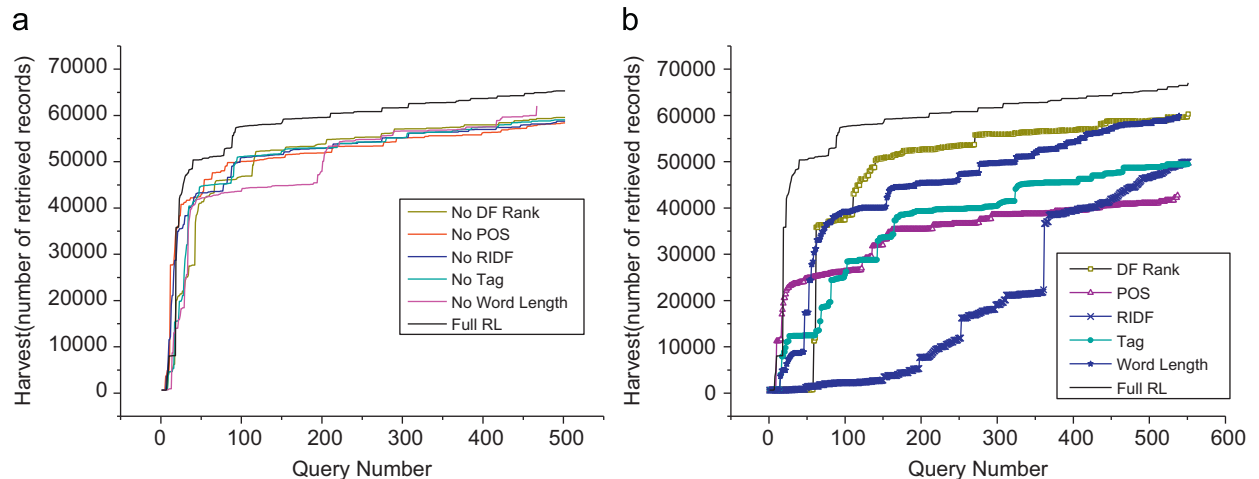
### 6.4. Compare the performance to the baseline methods

In recent years, many researchers, in both academia and industry, have developed their methods for deep web surfacing without the guidance of prior knowledge [12–18]. Based on the difference in estimation of query reward, these methods can be classified into the following three categories.

*Random* [12,13,17]: Perhaps it is the most straightforward solution to the problem. The reward of a query is randomly assigned a float. Formally, we have  $Q(s_t, a) = \text{random}(1.0)$ , in which  $\text{random}(1.0) \in [0,1]$  is a random generation function. The intuition of the method is that the random query is qualified enough to retrieve deep web records. It seems that the crawling process is accelerated since no further reward

**Table 12**  
Experimental results on the deep web sites.

| DB name      | Harvest   | Estimated database size | Coverage (%) | #Queries |
|--------------|-----------|-------------------------|--------------|----------|
| AbeBooks     | 90,224    | 110,000                 | 82.0         | 322      |
| Wikicfp      | 4,125     | 5,200                   | 79.3         | 499      |
| Paper Open   | 732,123   | 743,000                 | 98.5         | 425      |
| Yahoo movie  | 126,710   | 128,000                 | 99.0         | 367      |
| Google music | 85,378    | 110,000                 | 77.6         | 592      |
| Baidu        | 820,180   | 1,000,000               | 82.0         | 1950     |
| Baike        |           |                         |              |          |
| CiteSeer     | 1,013,214 | 1,121,821               | 90.3         | 53       |



**Fig. 2.** Feature analysis for the RL method. (a) Experiments by removing each feature and (b) experiments using only single feature.

estimation is needed, and update of the local statistics is also skipped. But unfortunately it is not the case at all times due to the inferior performance.

*Generic frequency* [13,17]: The reward of a keyword is evaluated by the generic document frequency of the keyword  $k$  in the acquired record set. Formally, we have  $Q(s_t, a) = DF(s_t, a)$ , in which  $DF(s_t, a)$  is the document frequency defined in Definition 4. The intuition is that the frequent keywords in the acquired record set ensures that more new records matching the keywords will be returned from the deep web database. The reward of a keyword varies from states to states making the reward estimation subtle and elusive.

According to the Definition 4, the reward of a query equals to the portion of new records retrieved by the query and it is quite likely that the new records retrieved by the same query may change, given the acquired records, i.e.,  $r(s_t, a) \neq r(s_j, a)$ . This is because the response set of new record is relevant to the states, i.e.,  $R_{\text{new}}(s_t, a) \neq R_{\text{new}}(s_j, a)$ . On the other hand, the response record set stays invariant in different states, i.e.,  $R(s_t, a) \equiv R(s_j, a)$ . We call it the state irrelevant property of response record set. As a consequence, instead of directly estimating subtle reward, many researchers thereafter turn to estimate the invariant part  $R(s_t, a)$  in the reward. And then the reward can be calculated using Eq. (4) shown as follows, which brings a new method – presumed distribution.

*Presumed distribution (Zipf)* [12,14,18]: Keywords distribution in the corpus has been well studied in academia. The most prestigious one among them is the Zipf–Mandelbrot law [29], which reveals a relationship between the words DF rank and the frequency  $f$  of a word in a text corpus. By making an assumption that the frequency equals the size of the response record set, i.e.,  $f = |R(s_t, a)|$ , Zipf–Mandelbrot law can be applied, i.e.,  $|R(s_t, a)| = |\alpha(r + \beta)^{-\gamma}|$ , in which  $\alpha$ ,  $\beta$  and  $\gamma$  are parameters. In each state, it is fitted using samples in the training set  $Tr$ , and is further used to evaluate instance in the candidate set  $C$ . Other distribution such as Gaussian distribution may also be exploited. Comparing to the random and genetic-frequency method, the presumed distribution method improves the performance by incorporating prior knowledge of the keywords distribution. However, according to our experimental observations, the performance of presumed distribution method becomes inferior in the following two cases. First, the target databases are multi-attribute, in which the presumed distribution does not hold in one certain query interface bound to a single attribute. Typical example includes the AbeBooks site. Second, the sites provide non full text search. Noisy words, template words and function words excluded from the index can mislead the distribution. In Baidu Baike, we found that quite a few issued words picked out by this method, most of which are template words and function words, actually retrieve no response record.

Besides, we have empirically studied the set covering sample [15] approach. In spite of having analyzed the deficiencies of the set covering sample method in the related work, we constructed full text index search engines using Lucene based on 70,000 pages from Yahoo movie and 1,000,000 pages from CiteSeer. And then we generated 20 queries that cover 99% of the 2000 sampling pages. Those 20 queries can harvest more than 80% of the total indexed data sets. However, a lot of those queries turned to be stop words and template words of Yahoo movie and CiteSeer, such as “and”, “the”, “or”, “Yahoo”, “Home”, “for”, etc., which can harvest fewer results.

It is interesting to note that RL is more general compared with the three baseline methods. Reexamine the Eq. (13), if future reward of an action is ignored, i.e.,  $h=0$  and the reward of an action is determined by a presumed distribution, the RL degenerates to the Zipf, i.e.,  $Q(s_t, a_i) = r(s_t, a_i)$ . Further reconsider the Eq. (4), if the acquired portion of an action is ignored too, i.e.,  $R(s_t, a_i) = \Phi$ , the RL degenerates to the GF (genetic-frequency), i.e.,  $Q(s_t, a_i) \propto DF(s_t, a_i)$ .

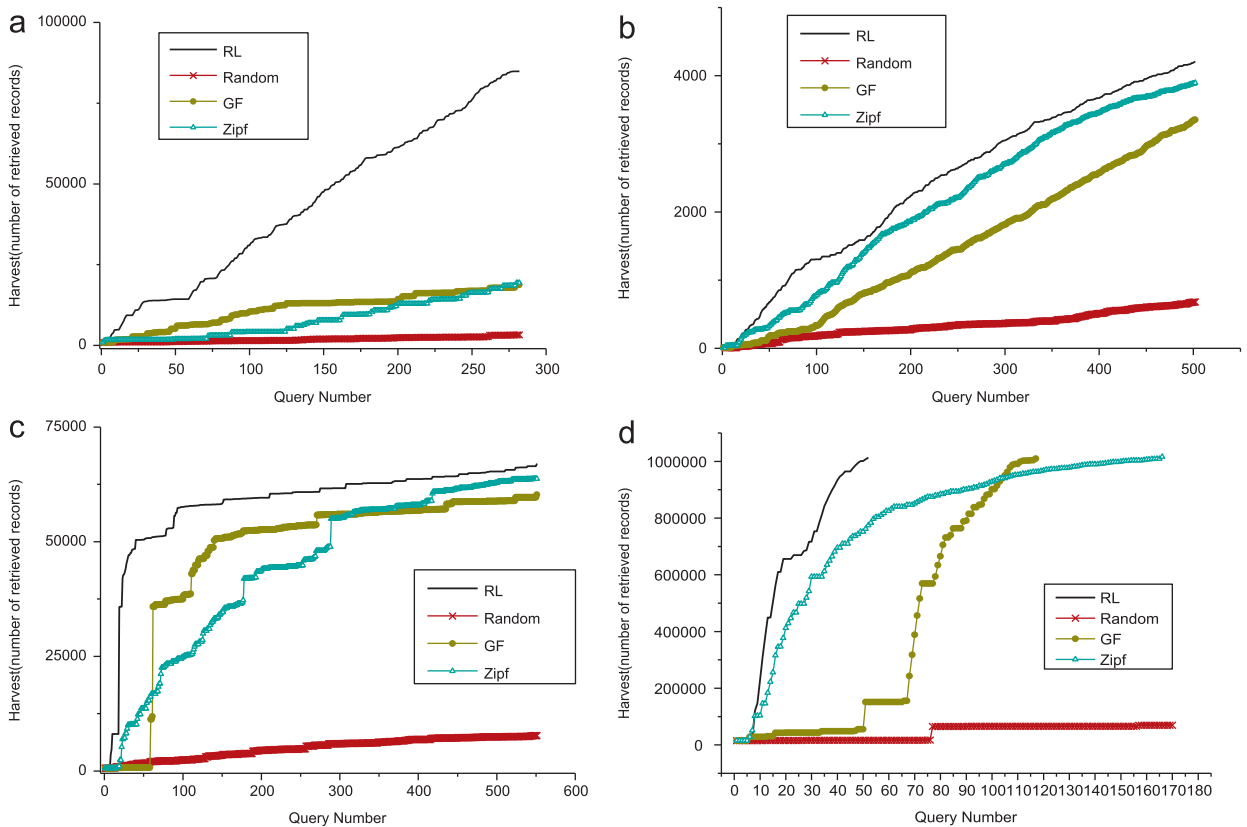
$$Q(s_t, a) = r(s_t, a) + \max \left[ \sum_{i=1}^h \gamma^i \times r(s_{t+i}, \pi(s_{t+i})) \right]. \quad (13')$$

$$r(s_t, a) = \frac{|R(s_t, a)| - DF(s_t, a)}{|D|}. \quad (4')$$

We performed our adaptive RL method as well as the baseline methods on Baidu Baike, Wikicfp, Yahoo Movie and CiteSeer. These four sites are chosen since they are representative sites in several aspects. Baidu Baike is the largest Wikipedia site in Chinese while Wikicfp has the smallest scale and is English. CiteSeer is a large scale digital library in English while Yahoo movie represents the medium scale movie-like databases. The experimental results are displayed in Fig. 3, in which the y-axis denotes the harvest, while the x-axis represents the query number. In the two sites, candidate sets of the methods choose actions by ranking DF and share the same size. In Baidu Baike and CiteSeer the size is 1000 and in Wikicfp and Yahoo Movie 500. All the four methods in each site start crawling from the same initial keyword. As we can see, the result shows that RL method is more efficient than baseline methods on the target websites. In Baidu Baike, the RL method achieves four times more coverage than the baseline methods do. In Wikicfp and Yahoo Movie it also outperforms the other three baselines although without that huge superiority. In CiteSeer, the superiority of RL is obvious. It reaches 90% coverage using only 53 queries, much faster than other three methods.

The RL method selects queries more relevant to the rewarding index fields, e.g. the ‘title’ field on the motivating example in Section 4. This suggests that the agent using RL learns the experience from its previous queries and hence sticks on the keywords matching against more records; whereas the agent using other methods do not make any adjustment when the presumed assumption is not applied. This well accounts for the slipping performance of Zipf method in Baidu Baike (Zipf does not hold in Baidu Baike). When referring to Fig. 4(a), we thus see a decreasing trend of reward queries. This proves the inherent advantage of RL





**Fig. 3.** Comparisons with three baselines on different deep web sites. (a) Experiment on Baidu Baike, (b) Experiment on Wikicfp, (c) Experiment on Yahoo Movie and (d) Experiment on CiteSeer.

method over the presumed distribution method. However, we found that the adjustment may occur in different periods, corresponding to the stagnation section in the growth curve in Fig. 3. In Baidu Baike the adjustment occurred during query 30–60, while in Wikicfp it occurred during the query 90–120 shown as a sharp increase in RL curve in Fig. 4(b). In this period, the RL method is confused by function words (like “the” and “on”) and ordinal numbers (such as “10th” and “15th”) that have not experienced before. These words appear very frequently not only in the title but also in the main body of a record page simply because conferences in Wikicfp virtually follow the name pattern like “the 10th international conference on ...” After the adjustment, the RL method turns its trend to continuous growth.

The RL method selects a keyword according to the “experience” learnt from the executed actions, thus it has a better awareness of the environment and leads to more accurate estimation for succeeding rewards. As we found in the experiments the rewarding keywords are issued earlier in RL than in other methods. For example, in both Baidu Baike and Wikicfp, their first 50 queries contain only about 20% non-reward queries (queries that get no reward), while other baselines contain more than 40%. In Fig. 4, we present the percentage of the non-reward queries under different periods with an interval of 50 queries. It clearly shows that the RL method has a significant lead in the number of reward queries,

indicating its better capability of avoiding non-responsive queries.

One may argue the non-responsive keywords can be ruled out beforehand by specifying a stop word list. However, sometimes stop words of one site could be very rewarding keywords for other sites. For example, function words and ordinal numbers are stop words in Wikicfp but turn out to be very rewarding in Yahoo movie. We list the first 20 queries of the four different approaches on Yahoo movie in Table 13. We can find there are no non-reward queries in RL while quite a few appear in the other three. After the 10th query, the RL method keeps choosing function words like “soon”, “all”, “this”, etc. and get very high reward. Because of this, we do not specify a stop word list before crawling. One may want to know whether the RL will keep its predominance if all the non-responsive keywords are wiped out. We conducted the comparison experiments on Yahoo movie by ruling out all the non-responsive words and found that the RL method still gains clear superiority over the other three methods. The results are shown in Fig. 6(b). All these demonstrate that RL method has solid capability of learning ‘pattern’ of rewarding keywords.

From what have been discussed above, further conclusions can be drawn about RL method. First, it may suffer from lacking of awareness of some complex environments in its initial period. For example, in Wikicfp whose indexing rules seem not so intelligible, it paid a price to

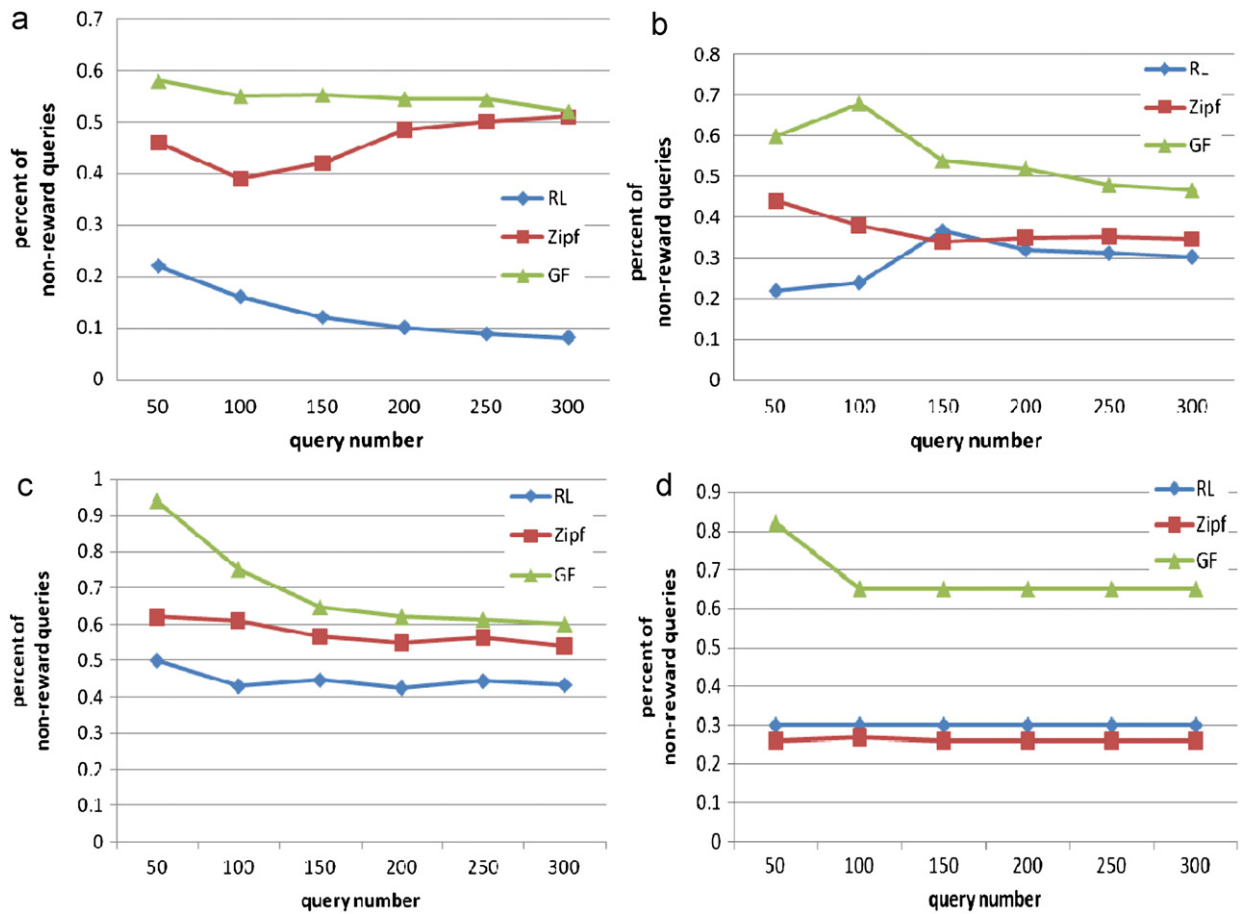


Fig. 4. Performance comparisons with Zipf and GF. (a) Baidu Baike, (b) Wikicfp, (c) Yahoo movie and (d) CiteSeer.

Table 13

Queries selection comparison between baselines and RL on Yahoo movie.

| Query# | Zipf       |         | GF            |         | Random      |         | RL       |         |
|--------|------------|---------|---------------|---------|-------------|---------|----------|---------|
|        | Keyword    | Harvest | Keyword       | Harvest | Keyword     | Harvest | Keyword  | Harvest |
| 1      | Trailers   | 629     | trailers      | 629     | trailers    | 629     | Trailers | 629     |
| 2      | Browse     | 30      | browse        | 30      | fulfill     | 0       | Browse   | 30      |
| 3      | Credits    | 1       | credits       | 1       | critical    | 46      | Credits  | 1       |
| 4      | What       | 8       | what          | 8       | minguez     | 71      | 2009     | 8       |
| 5      | Fresh      | 985     | fall          | 985     | tamas       | 0       | 2010     | 999     |
| 6      | 75         | 85      | search        | 465     | third       | 5       | 9        | 134     |
| 7      | Downloads  | 50      | entertainment | 656     | fighting    | 258     | Find     | 120     |
| 8      | Any        | 0       | user          | 85      | specializes | 341     | safe     | 974     |
| 9      | Lives      | 273     | movies        | 10      | playroom    | 15      | more     | 105     |
| 10     | Separated  | 949     | gossip        | 215     | hall        | 5       | Up       | 940     |
| 11     | Divorced   | 2       | astrology     | 18      | clerk       | 179     | soon     | 936     |
| 12     | Fiction    | 0       | resources     | 2       | nuanced     | 48      | All      | 705     |
| 13     | Different  | 103     | they          | 0       | julius      | 8       | This     | 899     |
| 14     | Shrinkwrap | 438     | top           | 997     | population  | 13      | The      | 906     |
| 15     | Qty        | 0       | rights        | 480     | hallmark    | 65      | See      | 992     |
| 16     | Four       | 0       | photos        | 87      | marni       | 8       | And      | 691     |
| 17     | Travel     | 844     | news          | 37      | fertitta    | 0       | To       | 960     |
| 18     | Try        | 229     | coming        | 136     | maqueen     | 0       | In       | 898     |
| 19     | Natural    | 389     | web           | 83      | hyde        | 0       | My       | 918     |
| 20     | Crime      | 142     | theaters      | 105     | prochnow    | 17      | Of       | 550     |

learn. However, this can be improved by incorporating with external knowledge or designing more sophisticated learning model. For instance, set some rules for picking up candidate queries to rule out those non-reward function words and ordinal words, or emphasize the POS feature for learning. Second, RL method loses its huge superiority when the target sites response a few in every query. If there are less difference between the most rewarding keyword and the worst one, then any method cannot perform much better than even random selection. In Wikicfp, the most rewarding query only get 46 new records. In this case, even a query sequence under the optimal crawling policy may not perform much better than Zipf or GF.

### 6.5. Running time analysis

We now analyze the running time of the different methods. During the experiments on the real world sites, the network condition varies in different times, making it difficult to calculate the precise time cost formulated in Section 3, i.e.,  $cost(s_t, a) = t_a + t_r \times |R(s_t, a)| + t_d \times |R_{new}(s_t, a)|$ .

$t_a$  is the cost of issuing an action, which include the query transmitting time and the query processing time by deep web databases.  $t_r$  is proportional to the average time of handling a response record.  $t_d$  is proportional to the average time of downloading a new result record. Based on this formulation, we believe that the cumulative running time of all the four methods will be proportional to the harvest. We also show  $t_a$  and  $t_r$  of every queries in the RL crawling process in Fig. 5(a) and (b) respectively. At the first beginning,  $t_a$  increases sharply since there are a lot of new keywords generated. After about 20 queries, it becomes to decrease because there are fewer and fewer new keywords. Finally it reaches a stable level where there are no more new keywords and the main time of  $t_a$  is updating the old candidate set and making a query submission.  $t_r$  is slowing decreasing because number of the new records needed to be processed is declining. But there are some exceptional points whose values are much higher than the regular trend due to the variety of file sizes.

However, it is easy to conclude that  $RL > Zipf > GF > Random$  in average time cost of a query through the comparison above, since to process the same amount of documents, the RL method need cost more time on

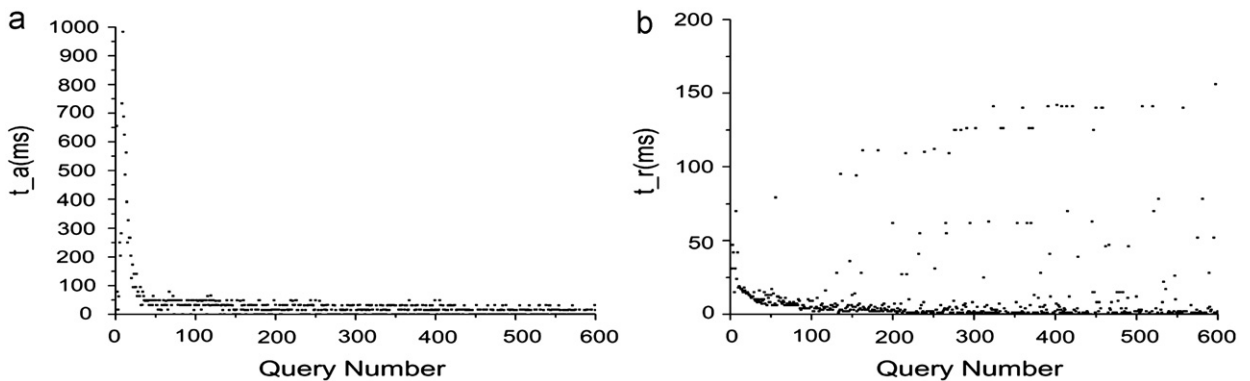


Fig. 5. Parameters of time cost in each query on Yahoo movie. (a) Time for issuing an action ( $t_a$ ) and (b) time for processing the response records ( $t_r$ ).

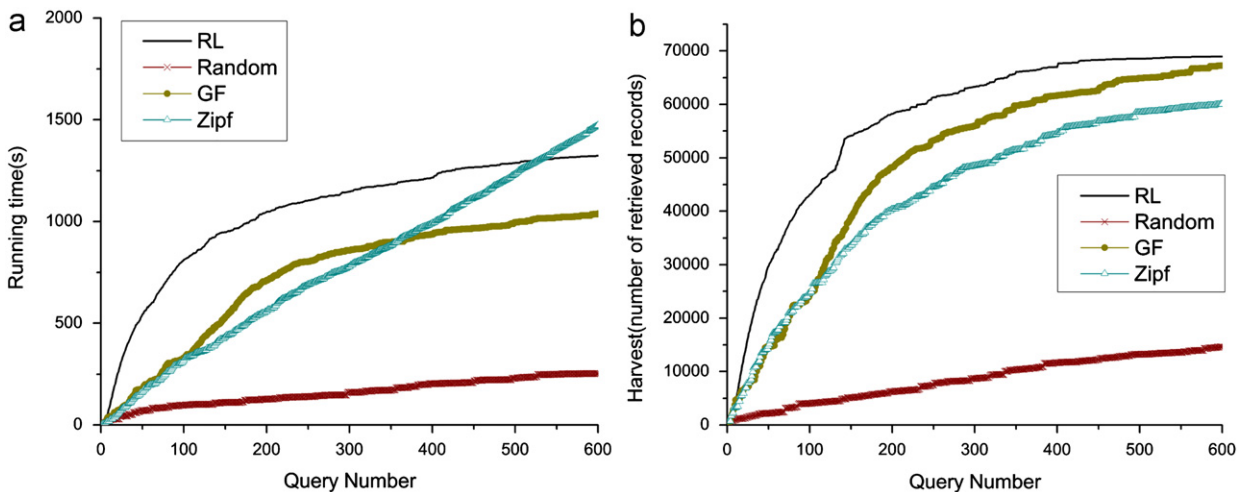


Fig. 6. Comparisons on Yahoo Movie. (a) Running time comparison and (b) harvest comparison by filtering out the stop words.

feature extraction and computation, while Zipf needs a little more time than other two to do Zipf-like data fitting. In Fig. 6(a) we present the accumulative running time of performing the different four methods on Yahoo Movie, in which  $x$ -axis represents the query number and  $y$ -axis denotes the running time in seconds. The experiments were conducted on the Intel Core 2 Duo CPU 3.00 GHz with 2 GB main memory and Window 7. The results curves demonstrate our conclusions. What supervised us is that Zipf's running time exceeds RL's after the 300th queries, which reproofed that the RL is not only an effective but also an efficient approach. What is more, from a search engine perspective, especially for those commercial search engines, coverage is more important than running time. Coverage is the essential metric to measure a search engine's quality, while the time cost can be easily reduced by technical approaches if money is not an issue. We give the three parameters in the cost function of the four methods on Yahoo movie in Table 14. Since  $t_d$  relies on the network condition, we give a range based on all our downloading time. The  $t_a$  and  $t_r$  are estimated using the average values over the whole process.  $t_a$  of RL method, although the largest one, but remains in the same level as Zipf and GF.

#### 6.6. Experiments on full text datasets

We also performed experiments on two benchmark web datasets, WebKB the 4 university dataset [41] and LDC2011T07 [42]. The first one contains 8282 web pages collected computer science departments of various universities. The second contains 254,418 English documents in New York Times Newswire Service. We built both full text index on these two datasets using Lucene and then run the comparison experiments on the four methods based on the same settings as in Section 6.4. The experimental results are presented in Fig. 7. We found that the RL method performs close to GF in both of the two datasets. In LDC the RL curve is almost the same to the GF curve so that it is totally covered by the GF curve. This

suggests that the RL method might lose its power and degenerate to GF on pure full text databases. However, the performance of Zipf differs on the two datasets. Zipf outperforms all the others in LDC but falls behind RL and GF in WebKB. We believe this is because that Zipf law is inclined to exist in large-scale and single-sourced datasets. The WebKB datasets are a small set of web pages gathered from different universities while the LDC is a much larger one from a single source 'New York Times Newswire Service'.

#### 6.7. Discussion

Finally, we give our suggestion to the applicability of the different crawling methods, i.e., what method should be employed for which type of deep web sites. In Table 15, the first four columns represent the different features of a deep web site and the last column lists the preference ranking. If a site is "Less rewarding", it responses very few result records for every query. Usually it also has a small size. "Multi-attributes" means there are more than one query interface provided by the site. For example, Abebooks provides "Author", "Title", "Keyword" and "ISBN" in its query form. "Single textbox" means that all queries can only be assigned to a single textbox interface. When the target site is "less rewarding" or provides full text search by a single textbox, the RL method loses its huge superiority over Zipf thus we suggest a higher rank for Zipf in these cases.

### 7. Conclusion and future work

In this paper we tackle the problem of deep web surfacing. The paper first presents a formal reinforcement learning framework to study the problem. We introduce an adaptive surfacing algorithm based on the proposed framework and propose its related methods for reward calculation and Q-value approximation. The framework enables a crawler to learn an optimized crawling strategy from the processed queries, and allows the crawler to make decisions on long term rewards. Experimental evaluation on 6 real world deep web sites shows that our method is both efficient and applicable. In general, it retrieves more than 80% of the total records by issuing a few hundreds of queries. When compared to the baseline methods, it shows a better performance in crawling coverage, promising query ratio, and running time, revealing a wider applicability to various deep web sites. Moreover, we delve into the crawling logs to analyze the superiority and imperfection of the RL method. Finally

**Table 14**  
Parameters of cost function on Yahoo movie.

| method | $t_a$ (ms) | $t_r$ (ms) | $t_d$ (ms) |
|--------|------------|------------|------------|
| RL     | 22         | 12         | 100–2000   |
| Zipf   | 10         | 208        | 100–2000   |
| GF     | 15         | 5          | 100–2000   |
| Random | 2          | 11         | 100–2000   |

**Table 15**  
Suggestions for choosing crawling methods.

| Unstructured Content | Less rewarding | Multi-attributes | Single textbox | Crawling method |
|----------------------|----------------|------------------|----------------|-----------------|
| –                    | Y              | –                | –              | Zipf > RL > GF  |
| N                    | N              | Y                | N              | RL > Zipf > GF  |
| N                    | N              | N                | Y              | RL > Zipf > GF  |
| Y                    | N              | Y                | N              | RL > Zipf > GF  |
| Y                    | N              | N                | Y              | Zipf > RL > GF  |

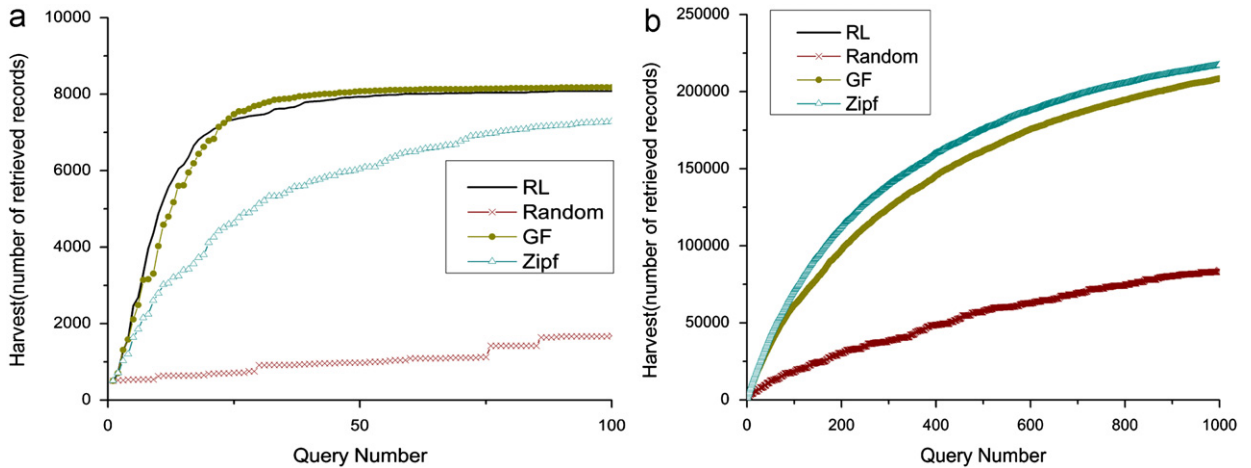


Fig. 7. Performance comparisons on benchmark datasets. (a) Experiment on WebKB and (b) experiment on LDC.

Table A1

The POS features used in RL method.

| ID | Attribute name           | Abbr. | Description                              |
|----|--------------------------|-------|------------------------------------------|
| 0  | NUMBER                   | Is    | List item marker                         |
|    |                          | cd    | Cardinal number                          |
| 1  | NOUN                     | NN    | Noun, singular or mass                   |
|    |                          | NNP   | Proper noun, singular                    |
|    |                          | NNPS  | Proper noun, plural                      |
|    |                          | NNS   | Noun, plural                             |
| 2  | ADJ                      | JJ    | Adjective                                |
|    |                          | JJR   | Adjective, comparative                   |
|    |                          | JJS   | Adjective, superlative                   |
| 3  | VERB_OR_MODAL            | MD    | Modal                                    |
|    |                          | VB    | Verb, base form                          |
|    |                          | VBD   | Verb, past tense                         |
|    |                          | VBG   | Verb, gerund or present participle       |
|    |                          | VBN   | Verb, past participle                    |
|    |                          | VBP   | Verb, non-3rd person singular present    |
|    |                          | VBZ   | Verb, 3rd person singular present        |
|    |                          | RB    | Adverb                                   |
| 4  | ADV                      | RBR   | Adverb, comparative                      |
|    |                          | RBS   | Adverb, superlative                      |
|    |                          | RP    | Particle                                 |
|    |                          | PDT   | Predeterminer                            |
| 5  | PNOUN_OR_PREDETERMINER   | POS   | Possessive ending                        |
|    |                          | PRP   | Personal pronoun                         |
|    |                          | PRP\$ | Possessive pronoun                       |
|    |                          | WDT   | Wh-determiner                            |
| 6  | W_WORD                   | WP    | Wh-pronoun                               |
|    |                          | WP\$  | Possessive wh-pronoun                    |
|    |                          | WRB   | Wh-adverb                                |
|    |                          | CC    | Coordinating conjunction                 |
| 7  | CONJ_OR_PREP_OR_TO_OR_DT | IN    | Preposition or subordinating conjunction |
|    |                          | TO    | to                                       |
|    |                          | DT    | Determiner                               |

we give the suggestive discussion to the applicability of crawling methods.

Although our RL method inherently consider the long term rewards and we demonstrated that considering long term reward may bring better results, we do not provide real world experimental results of RL method under  $h > 0$ .

We will consider using massive data tools and techniques such as Hadoop [36] and MapReduce [37] to implement it in the future. For example, calculating the DF value of each candidate query keywords is a typical MapReduce task. In addition, we do not consider how to apply the RL method to crawl deep web databases by querying



multiple attributes in this paper. All our presented experiments were performed on one textbox or single attribute of a query form. It will be worth of our future work to investigate how to extend the action  $a(k)$  from a simple keyword to a more general keyword vector for query forms. Similar work has been studied by us in [14] and it is possible to extend the RL method using the similar idea presented in our previous work.

## Acknowledgments

The research was supported in part by the National 863 Program of China under Grant no. 2012AA011003, the National Science Foundation of China under Grant nos. 60825202, 91118005, 61221063 and 91218301, the National Key Technologies R&D Program of China under Grant nos., 2011BAK08B01, 2011BAK08B02, 2012BAH16F02, 2013BAK09B01, the Doctoral Fund of Ministry of Education of China under Grant no. 20090201110060, Cheung Kong Scholar's Program. The authors are grateful to the anonymous reviewers for their comments which greatly improved the quality of the paper. Also thank Rui Li for his proof edit.

## Appendix A

See Table A1.

## References

- [1] M.K. Bergman, The deep web: surfacing hidden value, *The Journal of Electronic Publishing* 7 (2001) 3–21.
- [2] B. He, K. Patel, Z. Zhang, K.C.C. Chang, Accessing the deep web: A survey, *Communications of the ACM* 50 (5) (2007) 95–101.
- [3] J. Madhavan, S. Jeffery, S. Cohen, X. Dong, D. Ko, C. Yu, A. Halevy, Web-scale Data Integration: You Can Only Afford to Pay As You Go. In *Proceedings of CIDR2007*, pp. 342–350, 2007.
- [4] J. Madhavan, D. Ko, L. Kot, V. Ganapathy, A. Rasmussen, A., Halevy, Google's Deep-Web Crawl. In *Proceedings of VLDB2008*, Auckland, New Zealand, pp. 1241–1252, 2008.
- [5] J. Madhavan, L. Afanasiev, L. Antova, A., Halevy, Harnessing the Deep Web: Present and Future. In *Proceedings of CIDR*, Asilomar, CA, USA, 2009.
- [6] M. Shokouhi, L. Si, Federated Search, *Foundations and Trends in Information Retrieval* 5 (1) (2011) 1–102.
- [7] P.G. Ipeirotis, L. Gravano, Classification-aware hidden web text database selection, *ACM Transactions on Information Systems* 26 (2) (2008) 1–66.
- [8] H. He, W. Meng, C. Yu, Z. Wu Wise-integrator: an Automatic Integrator of Web Search Interfaces for E-commerce. In *Proceedings of VLDB2003*, Berlin, Germany, pp. 357–368, 2003.
- [9] K.C.C. Chang, B. He, Z. Zhang, Towards Large Scale Integration: Building a Metaquerier over Databases on the Web. In *Proceedings of CIDR*, Asilomar, CA, USA, 2005.
- [10] A.D. Sarma, X. Dong, A. Halevy, Bootstrapping Pay-As-You-Go Data Integration Systems. In *Proceedings of SIGMOD2008*, Vancouver, Canada, pp. 861–874, 2008.
- [11] S. Raghavan, H. Garcia-Molina, Crawling the Hidden Web. In *Proceedings of VLDB2001*, Rome Italy, pp. 129–138, 2001.
- [12] A. Ntoulas, P. Zefros, J. Cho, Downloading Textual Hidden Web Content through Keyword Queries. In *Proceedings of JCDL2005*, Denver, USA, pp. 100–109, 2005.
- [13] L. Barbosa, J. Freire, Siphoning Hidden-Web Data through Keyword-Based Interfaces. In *Proceedings of SBBD2004*, Brasilia, Brazil, pp. 309–321, 2004.
- [14] J. Liu, L. Jiang, Z.H. Wu, Q.H. Zheng, Deep Web adaptive crawling based on minimum executable pattern, *Journal of Intelligent Information Systems* 36 (2) (2011) 197–215.
- [15] Y. Wang, J.G. Lu, J. Liang, J. Chen, J. Liu, Selecting queries from sample to crawl deep web data sources, *Web Intelligence and Agent Systems* 10 (1) (2010) 75–88.
- [16] L. Jiang, Z.H. Wu, Q.H. Zheng, J. Liu, Learning Deep Web Crawling with Diverse Features. In *Proceedings of IEEE/WIC/ACM Web Intelligence*, Milan, Italy, pp. 572–575, 2009.
- [17] P. Wu, J.R. Wen, H. Liu, W.Y. Ma, Query Selection Techniques for Efficient Crawling of Structured Web Source. In *Proceedings of ICDE2006*, Atlanta GA, pp. 47–56, 2006.
- [18] P. Ipeirotis, L. Gravano, Distributed Search Over the Hidden Web: Hierarchical Database Sampling And Selection. In *Proceedings of VLDB2002*, Hong Kong, China, pp. 394–405, 2002.
- [19] C. Olston, M. Najork, Web Crawling, *Foundations and Trends in Information Retrieval* 4 (3) (2010) 175–246.
- [20] L.P. Kaelbling, M.L. Littman, A.W. Moore, Reinforcement learning: a survey, *Journal of Artificial Intelligence Research* 4 (1996) 237–285.
- [21] S. Raghavan, H. Garcia-Molina, Crawling the Hidden Web. In *Proceedings of VLDB2001*, Rome, Italy, pp. 129–138, 2001.
- [22] M. Alvarez, J. Raposo, A. Pan, F. Cacheda, F. Bellas, V. Carneiro, DeepBot: A Focused Crawler for Accessing Hidden Web Content. In *Proceedings of DEEC2007*, San Diego, CA, USA, pp. 18–25, 2007.
- [23] R.C. Sutton, A.G. Barto, *Reinforcement Learning: An Introduction*, The MIT Press, Cambridge, MA, 1998.
- [24] M. Yamamoto, K.W. Church, Using suffix arrays to compute term frequency and document frequency for all substrings in a corpus, *Computational Linguistics* 27 (1) (2001) 1–30.
- [25] C.J. Watkins, P. Dayan, Q-learning, *Machine Learning* 8 (1992) 279–292.
- [26] J. Ratsaby, Incremental learning with sample queries, *IEEE Transactions on Pattern Analysis and Machine Intelligence* 20 (8) (1998) 883–888.
- [27] S.C. Amstrup, T.L. McDonald, B.F.J. Manly, *Handbook of Capture–Recapture Analysis*, Princeton University Press, 2005.
- [28] H.S. Heaps, *Information Retrieval. Computational and Theoretical Aspects*, Academic Press, 1978, pp. 206–208.
- [29] B.B. Mandelbrot, *Fractal Geometry of Nature*, W. H. Freeman and Company, New York, 1988.
- [30] B.Y. Ricardo, R.N. Berthier, *Modern Information Retrieval*, ACM Press, 1999.
- [31] R. Levering, M. Cutler, *The Portrait of a Common HTML Web Page*. In *proceedings of DocEng 2006*, Amsterdam, Netherlands, pp. 198–204, 2006.
- [32] E. Hatcher, *O. Gospodnetic, Lucene in Action*, Manning Publication, 2004.
- [33] Toutanova Kristina, Manning D., Enriching the Knowledge Sources Used in a Maximum Entropy Part-of-Speech Tagger. In *Proceedings of the Joint SIGDAT Conference on Empirical Methods in Natural Language Processing and Very Large Corpora (EMNLP/VLC-2000)*, Hong Kong, pp. 63–70, 2000.
- [34] Toutanova, Kristina, Klein Dan, Manning Christopher, Singer Yoram, Feature-Rich Part-of-Speech Tagging with a Cyclic Dependency Network. In *Proceedings of HLT-NAACL*, Edmonton, Canada, pp. 252–259, 2003.
- [35] Suhit Gupta, Gail Kaiser, David Neistadt, Peter Grimm, DOM-based content extraction of HTML documents. In *Proceedings of WWW 2003*, New York, NY, USA, pp. 207–214, 2003.
- [36] <<http://hadoop.apache.org/>>.
- [37] J. Dean, S. Ghemawat, MapReduce: simplified data processing on large clusters, *Communications of the ACM* 51 (1) (2008) 107–113.
- [38] Z.H. Wu, L. Jiang, Q.H. Zheng, J. Liu, Learning to Surface Deep Web Content. In *Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence*, pp. 1967–1968, 2010.
- [39] L. Jiang, Z.H. Wu, Q. Feng, J. Liu, Q.H. Zheng, Efficient Deep Web Crawling Using Reinforcement Learning. In *Proceedings of PAKDD*, LNAI 6118, pp. 428–439, 2010.
- [40] P.G. Ipeirotis, E. Agichtein, P. Jain, L. Gravano, To Search or to Crawl?: Towards a Query Optimizer for Text-centric Tasks. In *Proceedings of SIGMOD*, Chicago, Illinois, USA, pp. 265–276, 2006.
- [41] <<http://www.cs.cmu.edu/afs/cs.cmu.edu/project/theo-20/www/data/>>.
- [42] Robert Parker, David Graff, Junbo Kong, Ke Chen, Kazuaki Maeda, *English Gigaword Fifth Edition*, Linguistic Data Consortium, Philadelphia, 2011.